

Rapport de projet de fin de module

FICHE DE SYNTHÈSE ACADÉMIQUE

Du perceptron aux modèles séquentiels : exploration des architectures fondamentales du Deep Learning avec PyTorch

Encadré par :

Pr.Mossab Batal

Réalisée par :

Belhachmi Hajar

Module :

Deep Learning

Filière :

Intelligence Artificielle et Science des Données

Année universitaire : 2025-2026

Résumé

Ce projet a été réalisé dans le cadre d'un module académique de **Deep Learning** afin d'étudier, d'implémenter et de comparer plusieurs architectures majeures de réseaux de neurones à l'aide de **PyTorch**. À travers une approche progressive, le travail explore successivement les perceptrons multicouches (MLP), les réseaux de neurones convolutionnels (CNN) et les réseaux de neurones récurrents (RNN), chacun étant appliqué à une problématique adaptée à sa nature.

La première partie est consacrée aux MLP et met en évidence les principes fondamentaux de l'apprentissage supervisé sur des données tabulaires. La seconde partie aborde les CNN à travers une tâche de classification d'images sur le jeu de données Fashion-MNIST, en étudiant notamment les opérations de convolution, de pooling et la visualisation des caractéristiques apprises. Enfin, la troisième partie traite des modèles séquentiels, depuis les RNN simples jusqu'aux architectures LSTM, GRU et Seq2Seq, appliquées à une tâche de traduction automatique.

Au-delà de l'implémentation des modèles, ce projet accorde une place importante à l'analyse des résultats, à l'expérimentation des hyperparamètres et à la comparaison des performances obtenues. Il permet ainsi de mieux comprendre les forces, les limites et les domaines d'application de chaque famille d'architectures, tout en développant des compétences pratiques en Deep Learning et en programmation avec PyTorch.

Abstract

This project was carried out as part of an academic **Deep Learning** course with the aim of studying, implementing, and comparing several fundamental neural network architectures using the **PyTorch** framework. Following a progressive learning approach, the work explores three major families of deep learning models: Multi-Layer Perceptrons (MLP), Convolutional Neural Networks (CNN), and Recurrent Neural Networks (RNN).

The first part focuses on MLPs and introduces the core principles of supervised learning applied to tabular data. The second part investigates CNNs through an image classification task using the Fashion-MNIST dataset, emphasizing convolution operations, pooling mechanisms, and the visualization of learned feature representations. The final part addresses sequential modeling by examining vanilla RNNs, LSTMs, GRUs, and Seq2Seq architectures within the context of machine translation tasks.

Beyond model implementation, this project highlights the importance of experimental analysis, hyperparameter exploration, and performance evaluation. The comparative study provides valuable insights into the strengths, limitations, and application domains of each architecture. Overall, this work contributes to a deeper understanding of deep learning concepts while strengthening practical skills in model development and experimentation with PyTorch.

Table des matières

Résumé	2
Abstract	3
Table des matières	4
Listes des figures	8
Listes des tableaux.....	9
Listes des abréviations.....	10
Introduction générale.....	11
1. Chapitre 1 : Étude des perceptrons multicouches (MLP).....	12
1.2.1. nn.Module — La classe de base de tout modèle.....	13
1.2.2. Paramètres et Gradients	13
1.2.3. Propagation Avant et Rétropropagation.....	14
1.2.4. Gestion du Device (CPU / GPU)	14
1.3.1. Présentation du Dataset Breast Cancer Wisconsin.....	14
1.3.2. Pipeline Complet de Préparation des Données	14
1.4.1. Version A — nn.Sequential : Architecture Déclarative.....	15
1.4.2. Version B — Classe Personnalisée : Architecture Contrôlée	15
1.5.1. named_parameters() — Inventaire des Paramètres Apprenables	16
1.5.2. state_dict() — État Complet du Modèle	16
1.6.1. Initialisation Gaussienne	16
1.6.2. Initialisation Constante — Brise la Symétrie.....	16
1.6.3. Initialisation Xavier Uniform — Recommandée	17
1.7.1. Fonction de Perte : CrossEntropyLoss.....	17
1.7.2. Optimiseur : Adam.....	17
1.7.3. Tableau des Hyperparamètres et Justifications	17
1.7.4. Boucle d'Entraînement Complète.....	18
1.8.1. Sauvegarde du Meilleur Modèle (Best Checkpoint).....	19
1.8.2. Rechargement et Vérification d'Identité	19
1.9.1. Métriques de Classification Binaire	19
1.9.2. Résultats Expérimentaux sur Breast Cancer Wisconsin	20
1.10.1. Pertinence Théorique : Le Théorème d'Approximation Universelle	22
1.10.2. Choix Méthodologiques Justifiés.....	22

1.10.3.	Résultats Expérimentaux et Comparaison	23
1.10.4.	Limites Principales.....	23
2.	Chapitre 2 : Réseaux de neurones convolutionnels (CNN)	24
2.2.1.	Présentation du Dataset.....	25
2.2.2.	Classes du Dataset.....	26
2.2.3.	Chargement et Normalisation	26
2.3.1.	Explosion du Nombre de Paramètres	27
2.3.2.	Perte de la Structure Spatiale	28
2.3.3.	Absence d'Invariance aux Translations.....	29
2.3.4.	Tableau Comparatif MLP vs CNN	29
2.4.1.	Localité (Local Connectivity)	29
2.4.2.	Partage des Poids (Weight Sharing).....	29
2.4.3.	Hierarchie des Représentations	30
2.5.1.	Corrélation Croisée 2D	30
2.5.2.	Calcul de la Taille de Sortie.....	31
2.5.3.	Padding et Stride	32
2.6.1.	Max-Pooling	32
2.6.2.	Average-Pooling	33
2.6.3.	Comparaison Max vs Average Pooling.....	33
2.7.1.	Corrélation Croisée 2D depuis Zéro	33
2.7.2.	Validation contre PyTorch	33
2.8.1.	LeNet-5 Original (LeCun, 1998)	34
2.8.2.	Modernisation pour Fashion-MNIST.....	34
2.8.3.	Flux Dimensionnel Complet	35
2.9.1.	Batch Normalization (Ioffe & Szegedy, 2015)	36
2.9.2.	Dropout (Srivastava et al., 2014)	37
2.9.3.	Initialisation Xavier Uniform (Glorot & Bengio, 2010)	37
2.10.1.	Fonction de Perte : Cross-Entropie	37
2.10.2.	Optimiseur : Adam + Weight Decay.....	37
2.10.3.	Scheduler : CosineAnnealingLR.....	38
2.10.4.	Boucle d'Entraînement Complète.....	38
2.11.1.	Impact du Padding	41
2.11.2.	Impact du Stride	42
2.11.3.	Convolution 1×1	42
2.12.1.	Feature Maps — Définition et Rôle	42

2.12.2.	Extraction via Hooks PyTorch.....	42
2.12.3.	Interprétation des Filtres Appris	44
2.13.1.	Résultats Expérimentaux Quantitatifs.....	45
2.13.2.	Analyse par Classe	46
2.13.3.	Justification Théorique de la Supériorité du CNN.....	47
3.	Chapitre 3 : Modélisation des données séquentielles avec les RNN	51
3.2.1.	Objectif probabiliste.....	52
3.2.2.	Log-probabilités et Negative Log-Likelihood	53
3.2.3.	Perplexité — Métrique Standard.....	54
3.2.4.	Approche N-grammes versus Modèles Neuraux	55
3.3.1.	Représentation one-hot	56
3.3.2.	Embeddings denses appris	56
3.4.1.	Idée centrale — La mémoire récurrente.....	57
3.4.2.	Équations fondamentales	57
3.4.3.	Partage des poids dans le temps.....	58
3.4.4.	Décompte des paramètres	58
3.5.1.	Rétropropagation à travers le temps (BPTT)	58
3.5.2.	Problèmes de gradient — Vanishing et Exploding.....	59
3.5.3.	Gradient Clipping — Solution contre l'Explosion	59
3.5.4.	BPTT Tronquée	60
3.6.1.	Motivation et Innovation.....	60
3.6.2.	Équations Complètes du LSTM.....	61
3.6.3.	Pourquoi c_t Résout le Gradient Évanescent.....	61
3.6.4.	Décompte des Paramètres	61
3.7.1.	Principe de Simplification.....	62
3.7.2.	Équations du GRU	62
3.7.3.	Comparaison Exhaustive LSTM / GRU	62
3.8.1.	RNN Profonds — Empilement de Couches.....	63
3.8.2.	Dropout Récurrent	64
3.8.3.	RNN Bidirectionnels.....	64
3.9.1.	Description du Dataset	64
3.9.2.	Pipeline de Prétraitement	64
3.9.3.	Padding, Masquage et Perte Masquée.....	66
3.10.1.	Limitation du Modèle de Langage Simple.....	66
3.10.2.	Architecture Encodeur-Décodeur.....	66

3.10.3.	Teacher Forcing	67
3.10.4.	Bottleneck du Vecteur de Contexte	68
3.11.1.	Formulation du Problème.....	68
3.11.2.	Décodage Glouton (Greedy Decoding).....	68
3.11.3.	Beam Search	68
3.12.1.	Deux Types de Métriques d'Évaluation	69
3.12.2.	Score BLEU — Bilingual Evaluation Understudy	69
3.12.3.	Limitations et Métriques Modernes	70
3.14.1.	Du Modèle de Langage au Seq2Seq — Généraliser à Deux Séquences.....	72
3.14.2.	Qualité du Décodage — Beam Search vs Glouton	73
3.14.3.	Limites et Perspectives — Vers les Transformers	73
	Conclusion Générale	76
	Références	77

Listes des figures

Figure 1 : Analyse comparative des performances des architectures MLP sur Breast Cancer Wisconsin..	21
Figure 2 : Exemples du dataset Fashion-MNIST par classe.....	26
Figure 3 : Comparaison du nombre de paramètres MLP vs CNN.....	28
Figure 4 : Application de filtres de détection sur une image Fashion-MNIST	31
Figure 5 : Courbes d'entraînement de LeNetModern	36
Figure 6 : Étude d'ablation : impact des choix architecturaux.....	39
Figure 7 : Comparaison des types de pooling	40
Figure 8 : Cartes de caractéristiques (feature maps) par couche	43
Figure 9 : Filtres appris de la couche Conv1	44
Figure 10 : Comparaison des courbes de convergence MLP vs CNN	45
Figure 11 : Matrices de confusion normalisées MLP vs CNN.....	47
Figure 12 : Synthèse visuelle finale (6 sous-graphiques).....	49
Figure 13 : Règle de chaîne : probabilités vs log-probabilités	53
Figure 14 : Analyse de la perplexité selon différents scénarios	55
Figure 15 : Visualisation des vecteurs d'embedding avant entraînement.....	57
Figure 16 : Effet du Gradient Clipping sur la norme des gradients.....	60
Figure 17 : Comparaison de la complexité paramétrique RNN vs GRU vs LSTM	63
Figure 18 : Distribution des longueurs de séquences (EN/FR)	65
Figure 19 : Entraînement du modèle Seq2Seq et curriculum de Teacher Forcing.....	67
Figure 20 : Précision des n-grammes et score BLEU par stratégie de décodage	70
Figure 21 : Comparaison complète RNN vs LSTM vs GRU	71
Figure 22 : Synthèse finale : profils radar et tableau comparatif des architectures	73

Listes des tableaux

Tableau 1 : Paramètres et gradients (Concept, définition, accès PyTorch)	13
Tableau 2 : Caractéristiques du dataset Breast Cancer Wisconsin	14
Tableau 3 : Avantages et limitations de nn.Sequential	15
Tableau 4 : Comparaison named_parameters() vs state_dict()	16
Tableau 5 : Comparaison des stratégies d'initialisation des poids	17
Tableau 6 : Hyperparamètres et justifications académiques	18
Tableau 7 : Méthodes de sauvegarde (state_dict vs modèle complet)	19
Tableau 8 : Métriques de classification binaire	20
Tableau 9 : Résultats expérimentaux sur Breast Cancer Wisconsin	20
Tableau 10 : Caractéristiques du dataset Fashion-MNIST	25
Tableau 11 : Classes du dataset Fashion-MNIST (10 classes)	26
Tableau 12 : Comparaison MLP vs CNN (critères multiples)	29
Tableau 13 : Progression des représentations apprises au sein d'un CNN	30
Tableau 14 : Calcul des tailles de sortie (flux dimensionnel LeNet)	32
Tableau 15 : Rôles de padding et stride	32
Tableau 16 : Comparaison Max-Pooling vs Average-Pooling (exemple 4×4)	33
Tableau 17 : Validation des implémentations manuelles vs PyTorch	34
Tableau 18 : Composant LeNet-5 original vs LeNetModern	34
Tableau 19 : Flux dimensionnel complet LeNetModern	35
Tableau 20 : Étude expérimentale des hyperparamètres (9 configurations)	39
Tableau 21 : Comparaison MLP vs CNN (résultats quantitatifs)	45
Tableau 22 : Formulaire de référence CNN	49
Tableau 23 : Tableau des perplexités et interprétations	54
Tableau 24 : Symboles et dimensions du RNN vanilla	58
Tableau 25 : Équations complètes du LSTM (6 portes)	61
Tableau 26 : Équations du GRU (4 composantes)	62
Tableau 27 : Comparaison exhaustive LSTM vs GRU	62
Tableau 28 : Tokens spéciaux utilisés dans l'architecture du modèle	65
Tableau 29 : Teacher Forcing vs Auto-régressif	67
Tableau 30 : Comparaison des stratégies de décodage (Glouton vs Beam Search)	69
Tableau 31 : Comparaison globale des architectures (RNN, LSTM, GRU, Seq2Seq...)	71

Listes des abréviations

- **MLP** : Multi-Layer Perceptron (Perceptron Multicouche)
- **CNN** : Convolutional Neural Network (Réseau de Neurones Convolutionnel)
- **RNN** : Recurrent Neural Network (Réseau de Neurones Récurrent)
- **LSTM** : Long Short-Term Memory
- **GRU** : Gated Recurrent Unit
- **Seq2Seq** : Sequence to Sequence
- **DL** : Deep Learning
- **NLP** : Natural Language Processing (Traitement Automatique du Langage Naturel)
- **BPTT** : Backpropagation Through Time
- **MLE** : Maximum Likelihood Estimation
- **NLL** : Negative Log-Likelihood
- **PPL** : Perplexity (Perplexité)
- **LM** : Language Model (Modèle de Langage)
- **TF** : Teacher Forcing
- **BLEU** : Bilingual Evaluation Understudy
- **NMT** : Neural Machine Translation
- **NER** : Named Entity Recognition
- **POS** : Part-of-Speech Tagging
- **OOV** : Out-of-Vocabulary
- **GAP** : Global Average Pooling
- **BN** : Batch Normalization
- **ReLU** : Rectified Linear Unit
- **XAI** : Explainable Artificial Intelligence
- **SHAP** : SHapley Additive exPlanations
- **LIME** : Local Interpretable Model-agnostic Explanations
- **PCA** : Principal Component Analysis
- **SVM** : Support Vector Machine
- **GPU** : Graphics Processing Unit
- **CPU** : Central Processing Unit
- **BP** : Brevity Penalty
- **SOS** : Start of Sequence
- **EOS** : End of Sequence
- **PAD** : Padding Token
- **UNK** : Unknown Token
- **UCI** : University of California Irvine Machine Learning Repository

Introduction générale

L'essor considérable des données numériques et l'augmentation des capacités de calcul ont favorisé le développement de l'intelligence artificielle, et plus particulièrement du Deep Learning. Cette branche de l'apprentissage automatique repose sur l'utilisation de réseaux de neurones artificiels composés de plusieurs couches capables d'apprendre automatiquement des représentations complexes à partir des données. Grâce à leurs performances remarquables, ces modèles sont aujourd'hui largement utilisés dans de nombreux domaines tels que la vision par ordinateur, le traitement automatique du langage naturel, la santé, la finance ou encore les systèmes de recommandation.

Dans ce contexte, la maîtrise des principales architectures de Deep Learning constitue une étape essentielle dans la formation des futurs ingénieurs en informatique et en science des données. Cependant, comprendre le fonctionnement interne de ces modèles ne se limite pas à leur simple utilisation. Il est nécessaire d'en étudier les fondements théoriques, d'en réaliser l'implémentation pratique et d'analyser leurs comportements face à différents types de données et de problématiques.

Le présent projet, réalisé dans le cadre du module de Deep Learning, s'inscrit dans cette démarche pédagogique. Son objectif principal est d'étudier, d'implémenter et de comparer trois grandes familles d'architectures de réseaux de neurones à l'aide du framework PyTorch : les perceptrons multicouches (MLP), les réseaux de neurones convolutionnels (CNN) et les réseaux de neurones récurrents (RNN). Chacune de ces architectures répond à des besoins spécifiques et possède des caractéristiques propres qui la rendent plus adaptée à certains types de données.

La première partie du projet est consacrée aux MLP, qui constituent l'une des architectures les plus fondamentales du Deep Learning. Cette étude permet d'aborder les concepts essentiels liés à l'apprentissage supervisé sur des données tabulaires, notamment la propagation avant, la rétropropagation du gradient et l'optimisation des paramètres du modèle. La deuxième partie porte sur les CNN, reconnus pour leur efficacité dans les tâches de vision par ordinateur. À travers le jeu de données Fashion-MNIST, cette section met en évidence le rôle des opérations de convolution et de pooling dans l'extraction automatique de caractéristiques pertinentes à partir des images. Enfin, la troisième partie s'intéresse aux modèles récurrents et aux architectures Seq2Seq, utilisées pour le traitement des données séquentielles. Cette étude couvre les RNN classiques ainsi que leurs variantes améliorées, telles que les LSTM et les GRU, appliquées à une tâche de traduction automatique.

Au-delà de l'implémentation des modèles, ce projet accorde une attention particulière à l'expérimentation, à l'analyse des performances et à la comparaison des différentes approches étudiées. Les résultats obtenus permettent de mettre en évidence les avantages, les limites et les domaines d'application de chaque architecture. Ainsi, ce travail offre une vision globale des concepts fondamentaux du Deep Learning tout en développant des compétences pratiques indispensables à la conception et à l'évaluation de solutions d'intelligence artificielle modernes.



1. Chapitre 1 : Étude des perceptrons multicouches (MLP)

1.1. Objectifs de la Fiche

Cette fiche de synthèse constitue un document de référence académique couvrant l'ensemble des concepts et compétences requis dans la Partie I du projet de fin de module Deep Learning à l'EMSI. Elle est structurée selon la progression pédagogique du cours, allant des fondamentaux de PyTorch jusqu'à l'évaluation rigoureuse d'un modèle de réseau de neurones multicouche (MLP) sur un jeu de données clinique réel.

À l'issue de la lecture de cette fiche, l'étudiant sera en mesure de :

- Comprendre et implémenter un modèle PyTorch via `nn.Module`
- Distinguer `nn.Sequential` d'une classe personnalisée héritant de `nn.Module`
- Gérer les paramètres du modèle via `named_parameters()` et `state_dict()`
- Appliquer trois stratégies d'initialisation des poids : gaussienne, constante et Xavier
- Sauvegarder et recharger le meilleur modèle avec `torch.save` / `torch.load`
- Exécuter le modèle sur le device disponible (CPU / GPU) de manière cohérente
- Évaluer rigoureusement les performances : accuracy, precision, recall, F1-score, matrice de confusion

1.2. Concepts Fondamentaux de PyTorch

1.2.1. `nn.Module` — La classe de base de tout modèle

`nn.Module` est la classe parente de tous les modèles PyTorch. Elle fournit les mécanismes d'enregistrement automatique des paramètres apprenables, l'infrastructure de la méthode `forward()` à surcharger, les utilitaires de sauvegarde, de déplacement sur device (CPU/GPU), et la gestion des modes d'entraînement et d'évaluation. Tout réseau de neurones PyTorch, qu'il soit simple ou complexe, doit hériter de `nn.Module`.

1.2.2. Paramètres et Gradients

Un `nn.Parameter` est un tenseur dont l'attribut `requires_grad` est positionné à `True`. PyTorch construit dynamiquement un graphe de calcul (computational graph) lors de chaque propagation avant, ce qui permet de dériver automatiquement la perte par rapport à chaque paramètre appris via la règle de chaîne (algorithme de rétropropagation).

Concept	Définition mathématique	Accès PyTorch
Paramètre θ	Variable apprenable du modèle (W, b)	<code>module.weight</code> / <code>module.bias</code>
Gradient $\partial L / \partial \theta$	Sensibilité de la perte au paramètre	<code>module.weight.grad</code>
Valeur numérique	Tenseur des valeurs courantes	<code>module.weight.data</code>
<code>state_dict()</code>	Dictionnaire {nom $\rightarrow$ tenseur} du modèle	<code>model.state_dict()</code>

Tableau 1 : Paramètres et gradients (Concept, définition, accès PyTorch)

1.2.3. Propagation Avant et Rétropropagation

La boucle d'apprentissage PyTorch suit invariablement un schéma en cinq étapes séquentielles. Cette séquence est fondamentale : toute déviation produit des mises à jour de paramètres incorrectes.

Remarque importante : `zero_grad()` est indispensable car PyTorch accumule les gradients par défaut (comportement utile pour les architectures récurrentes). Sans cet appel, les gradients de plusieurs mini-batches s'additionnent et la mise à jour des poids diverge.

1.2.4. Gestion du Device (CPU / GPU)

PyTorch impose une règle absolue : tous les tenseurs impliqués dans une même opération mathématique doivent résider sur le même device. Le non-respect de cette règle lève immédiatement une `RuntimeError`.

1.3. Préparation et Ingénierie des Données

1.3.1. Présentation du Dataset Breast Cancer Wisconsin

Le Breast Cancer Wisconsin Dataset est un jeu de données clinique de référence, largement utilisé en apprentissage automatique pour la classification binaire. Il est intégré nativement dans la bibliothèque scikit-learn et provient du UCI Machine Learning Repository. Chaque observation correspond à l'analyse numérique d'une biopsie de tumeur mammaire.

Caractéristique	Détail
Source	UCI Machine Learning Repository — intégré dans scikit-learn
Nombre d'exemples	569 (357 bénins, 212 malins)
Nombre de features	30 mesures numériques continues (rayon, texture, périmètre, aire, concavité...)
Tâche	Classification binaire : 0 = malin (cancéreux) / 1 = bénin
Déséquilibre des classes	63 % bénins, 37 % malins — déséquilibre modéré à surveiller

Tableau 2 : Caractéristiques du dataset Breast Cancer Wisconsin

1.3.2. Pipeline Complet de Préparation des Données

La préparation rigoureuse des données est une étape critique en apprentissage supervisé. Trois opérations principales sont nécessaires : le découpage stratifié, la normalisation et la conversion en tenseurs PyTorch.

L'argument `stratify=y` garantit que la proportion de chaque classe est préservée dans chaque sous-ensemble. Cette précaution est essentielle sur des jeux de données déséquilibrés pour éviter que le set de test soit dominé par une seule classe, ce qui biaiserait l'évaluation.

Le `StandardScaler` applique la transformation suivante pour chaque feature j : $x'_j = (x_j - \mu_j) / \sigma_j$, où μ_j est la moyenne et σ_j l'écart-type calculés uniquement sur l'ensemble d'entraînement. Sans normalisation, les features à grande magnitude (exemple : aire cellulaire ≈ 1000) dominent les gradients, ralentissent la convergence et peuvent déstabiliser l'optimisation.

Data leakage : Appliquer fit_transform sur val/test avant le split contaminerait les ensembles de test avec les statistiques d'entraînement, produisant une évaluation artificiellement optimiste et non représentative des performances réelles sur données inconnues.

L'utilisation de DataLoader présente deux avantages majeurs : le batching automatique divise le dataset en mini-batches pour un apprentissage stochastique efficace, et le shuffle à chaque époque brise les corrélations séquentielles qui pourraient biaiser l'apprentissage.

1.4. Construction du Multilayer Perceptron

Un Multilayer Perceptron (MLP), ou réseau de neurones entièrement connecté (fully connected), est constitué de couches successives de neurones artificiels. Chaque neurone calcule une combinaison linéaire de ses entrées, suivie d'une fonction d'activation non-linéaire. Cette non-linéarité est ce qui confère au MLP sa capacité à apprendre des frontières de décision complexes (théorème d'approximation universelle, Cybenko 1989).

1.4.1. Version A — nn.Sequential : Architecture Déclarative

nn.Sequential enchaîne les modules dans l'ordre de déclaration. La méthode forward() est générée automatiquement : chaque couche reçoit en entrée la sortie de la couche précédente. Cette approche est idéale pour les architectures linéaires simples et le prototypage rapide.

Avantages	Limitations	Cas d'usage recommandé
Code très concis et lisible	Pas de skip connections ni branches parallèles	Architectures linéaires simples
Pas besoin de définir forward()	Accès difficile aux activations intermédiaires	Prototypage rapide et expérimentation

Tableau 3 : Avantages et limitations de nn.Sequential

1.4.2. Version B — Classe Personnalisée : Architecture Contrôlée

L'héritage de nn.Module offre un contrôle total sur l'architecture. Cette version intègre le BatchNorm1d (normalisation par lots) pour réduire le covariate shift interne et stabiliser l'entraînement. La méthode forward() explicite permet d'implémenter n'importe quelle topologie de réseau, y compris les connexions résiduelles, les sorties multiples ou les branches conditionnelles.

- **Rôle du BatchNorm1d**

BatchNorm1d normalise les activations d'une couche au sein d'un mini-batch selon la formule : $x'_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \times \gamma + \beta$ où μ_B et σ_B^2 sont la moyenne et la variance du batch courant, et γ , β sont des paramètres

apprenables. Cette normalisation : (1) réduit le covariate shift interne entre les couches, (2) accélère la convergence en permettant des learning rates plus élevés, et (3) agit comme régularisateur implicite, réduisant la dépendance au Dropout.

- **Rôle du Dropout**

Le Dropout, introduit par Srivastava et al. (2014), désactive aléatoirement une fraction p des neurones lors de chaque forward pass en entraînement. Cela force le réseau à développer des représentations redondantes

et robustes. En inférence (`model.eval()`), le Dropout est désactivé et toutes les connexions sont maintenues actives, avec une mise à l'échelle implicite des activations.

1.5. Inspection et Gestion des Paramètres

1.5.1. `named_parameters()` — Inventaire des Paramètres Apprenables

La méthode `named_parameters()` retourne un itérateur sur tous les paramètres dont `requires_grad=True`. Elle est indispensable pour diagnostiquer l'architecture, calculer la complexité du modèle, et appliquer des stratégies d'initialisation sélectives.

1.5.2. `state_dict()` — État Complet du Modèle

La méthode `state_dict()` retourne un dictionnaire ordonné contenant tous les tenseurs du modèle, y compris les paramètres apprenables et les buffers non-apprenables (statistiques `running_mean` et `running_var` de BatchNorm). C'est l'objet canonique à sauvegarder pour la persistance du modèle.

Méthode	Contenu	Usage principal
<code>named_parameters()</code>	Uniquement <code>requires_grad=True</code> (apprenables)	Inspection, debug, initialisation ciblée
<code>state_dict()</code>	Tous les buffers (poids, biais, BatchNorm stats)	Sauvegarde et rechargement du modèle

Tableau 4 : Comparaison `named_parameters()` vs `state_dict()`

1.6. Stratégies d'Initialisation des Poids

L'initialisation des poids est une étape critique et souvent sous-estimée de la conception d'un réseau de neurones. Une mauvaise initialisation peut provoquer deux pathologies : le *vanishing gradient* (les gradients deviennent ≈ 0 dans les premières couches, bloquant l'apprentissage) ou l'*exploding gradient* (les gradients divergent vers l'infini, rendant l'optimisation impossible). Une initialisation appropriée garantit que les activations et les gradients maintiennent une variance raisonnable tout au long du réseau.

1.6.1. Initialisation Gaussienne

Cette stratégie initialise chaque poids W_{ij} en tirant un échantillon d'une distribution normale de moyenne 0 et d'écart-type $\sigma = 0.01$. Les biais sont initialisés à 0.

Analyse : Maintient les poids petits au départ, ce qui est bénéfique pour la stabilité numérique initiale. Cependant, si σ est trop faible, les activations s'amortissent successivement à travers les couches et l'apprentissage stagne (*vanishing gradient*). Si σ est trop élevé, les activations explosent.

1.6.2. Initialisation Constante — Brise la Symétrie

Cette stratégie initialise tous les poids à une valeur constante c . Bien qu'elle puisse sembler simple et contrôlable, elle constitue une mauvaise pratique pour l'entraînement réel.

Problème de symétrie : Lorsque tous les neurones d'une couche sont initialisés identiquement, ils reçoivent les mêmes gradients et effectuent exactement les mêmes mises à jour. Ils restent donc identiques pendant toute la durée de l'entraînement, la couche est réduite à un seul neurone effectif. Cette initialisation est uniquement utile à des fins pédagogiques pour illustrer le problème de symétrie.

1.6.3. Initialisation Xavier Uniform — Recommandée

L'initialisation de Glorot & Bengio (2010), dite Xavier, adapte la variance des poids à la dimensionnalité des couches adjacentes. Pour une couche avec n_{in} entrées et n_{out} sorties, les poids sont tirés d'une distribution uniforme :

$$W \sim U\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right)$$

L'intuition mathématique est la suivante : si les poids sont trop grands ou trop petits, la variance des activations croît ou décroît exponentiellement avec la profondeur. Xavier maintient $\text{Var}(a_l) \approx \text{Var}(a_{l-1})$ à travers les couches, assurant des gradients de magnitude comparable du début à la fin du réseau.

Stratégie	std observé (fc1)	Risque vanishing	Recommandé
Gaussienne ($\sigma=0.01$)	≈ 0.0100	Modéré si σ faible	Cas spécifiques
Constante ($c=0.01$)	0.0000 (std=0)	Problème de symétrie	Non — pédagogique uniquement
Xavier Uniform	≈ 0.1455	Minimal	Oui — standard actuel

Tableau 5 : Comparaison des stratégies d'initialisation des poids

1.7. Entraînement du Modèle

1.7.1. Fonction de Perte : CrossEntropyLoss

Pour la classification multi-classes, PyTorch fournit `nn.CrossEntropyLoss`, qui combine de manière numériquement stable `LogSoftmax` et la perte log-vraisemblance négative (NLLLoss). La formule pour C classes et un batch de N exemples est :

$$L = -\frac{1}{N} \sum_{i=1}^N \log \left(\frac{\exp(z_{i,y_i})}{\sum_{c=1}^C \exp(z_{i,c})} \right)$$

Cette formulation attend des logits bruts en entrée (non-normalisés), et ne nécessite donc pas l'application d'une softmax en fin de réseau. Les labels doivent être des entiers de type `torch.long` (`dtype=torch.int64`).

1.7.2. Optimiseur : Adam

L'optimiseur Adam (Adaptive Moment Estimation, Kingma & Ba, 2014) maintient des taux d'apprentissage adaptatifs par paramètre en combinant les avantages de RMSProp et du momentum. Il calcule des estimations du premier moment (moyenne des gradients) et du second moment (variance des gradients) :

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t + \epsilon}} \quad \text{avec correction du biais}$$

1.7.3. Tableau des Hyperparamètres et Justifications

Hyperparamètre	Valeur	Justification académique
Fonction de perte	CrossEntropyLoss	Standard pour classification multi-classes, numériquement stable
Optimiseur	Adam	Adaptatif, convergence rapide, robuste au choix initial du LR
Learning rate η	1e-3	Valeur par défaut recommandée par Kingma & Ba (2014)
Weight decay λ	1e-4	Régularisation L2 : pénalise $\ \theta\ ^2$, réduit l'overfitting
Batch size	32	Compromis : gradients stables + régularisation implicite du mini-batch SGD
Early stopping	patience=15 époques	Arrêt si val_loss ne diminue pas $\rightarrow$ préserve la généralisation
LR Scheduler	ReduceLROnPlateau $\times 0.5$	Affinage de la convergence en réduisant le LR sur plateau

Tableau 6 : Hyperparamètres et justifications académiques

1.7.4. Boucle d'Entraînement Complète

Le modèle est entraîné en utilisant la fonction de perte **Cross-Entropy Loss**, adaptée aux problèmes de classification, et l'optimiseur **Adam** avec un taux d'apprentissage initial de **0,001** ainsi qu'une régularisation **L2 (weight decay = 10^{-4})** afin de limiter le surapprentissage. Un ordonnanceur de taux d'apprentissage (**ReduceLROnPlateau**) est également utilisé pour ajuster automatiquement le learning rate lorsque la perte de validation cesse de diminuer pendant **7 époques consécutives**, en le réduisant d'un facteur de **0,5**. À chaque époque, le modèle passe d'abord en mode entraînement (`model.train()`), où les couches **Dropout** et **Batch Normalization** fonctionnent en mode apprentissage. Pour chaque mini-lot de données, les gradients sont réinitialisés, une propagation avant est effectuée pour calculer les prédictions, la perte est évaluée, puis la rétropropagation calcule les gradients avant que l'optimiseur ne mette à jour les poids du réseau. Une phase de validation est ensuite réalisée en mode évaluation (`model.eval()`), sans calcul des gradients (`torch.no_grad()`), afin de mesurer la perte sur les données de validation. Enfin, un mécanisme d'**Early Stopping** est mis en place : le meilleur modèle, correspondant à la plus faible perte de validation, est sauvegardé, et si aucune amélioration n'est observée pendant **15 époques consécutives**, l'entraînement est interrompu automatiquement pour éviter le surapprentissage et réduire le temps de calcul.

model.train() vs model.eval() : En mode `train()`, Dropout masque aléatoirement des neurones et BatchNorm utilise les statistiques du mini-batch courant. En mode `eval()`, Dropout est intégralement désactivé et BatchNorm utilise les moyennes glissantes (`running_mean`, `running_var`) accumulées lors de l'entraînement.

1.8. Sauvegarde et Rechargement du Modèle

La sauvegarde du modèle est une pratique essentielle pour conserver le meilleur modèle obtenu au cours de l'entraînement et le réutiliser en production ou pour des expériences futures. PyTorch offre deux approches, mais la sauvegarde du `state_dict` est fortement recommandée.

1.8.1. Sauvegarde du Meilleur Modèle (Best Checkpoint)

À la fin de l'entraînement, les paramètres du modèle ayant obtenu la **plus faible perte de validation** sont sauvegardés dans un fichier nommé **best_model.pth** à l'aide de la fonction `torch.save()`. Cette sauvegarde permet de conserver la meilleure version du réseau de neurones, plutôt que les paramètres de la dernière époque, qui pourraient être moins performants en raison d'un éventuel surapprentissage. La valeur de la meilleure perte de validation (`best_val_loss`) est également affichée afin de fournir une indication des performances du modèle retenu.

1.8.2. Rechargement et Vérification d'Identité

Afin de vérifier que le modèle sauvegardé peut être réutilisé correctement, une nouvelle instance de l'architecture du réseau de neurones est d'abord recrée avec les mêmes paramètres que le modèle initial. Les poids précédemment sauvegardés dans le fichier **best_model.pth** sont ensuite chargés à l'aide de `load_state_dict()`. Le modèle est alors placé en **mode évaluation** (`model.eval()`), ce qui désactive les couches **Dropout** et utilise les statistiques mémorisées de **Batch Normalization**, garantissant ainsi des prédictions cohérentes. Enfin, les prédictions du modèle rechargé sont comparées à celles du modèle original sur l'ensemble de test. Une instruction `assert` vérifie qu'elles sont **strictement identiques**, confirmant que la sauvegarde et le rechargement des paramètres ont été effectués sans aucune perte d'information et que le modèle peut être déployé ou réutilisé de manière fiable.

Méthode de sauvegarde	Avantages	Inconvénients
<code>torch.save(state_dict)</code>	Léger, portable, robuste aux refactorisations du code	Nécessite de recréer manuellement l'architecture
<code>torch.save(model)</code>	Simple, tout-en-un	Dépend des imports et de la structure exacte du code Python

Tableau 7 : Méthodes de sauvegarde (*state_dict* vs modèle complet)

1.9. Évaluation des Performance

1.9.1. Métriques de Classification Binaire

L'évaluation d'un classifieur ne doit pas se limiter à l'accuracy, particulièrement en présence de déséquilibre de classes ou de coûts asymétriques entre les types d'erreurs. En diagnostic médical, un faux négatif (cancer non détecté) a des conséquences bien plus graves qu'un faux positif.

Métrique	Formule	Interprétation clinique
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Proportion globale de bonnes prédictions
Precision	$\frac{TP}{TP + FP}$	Parmi les prédits positifs, combien le sont vraiment ?

Métrique	Formule	Interprétation clinique
Recall (Sensibilité)	$\frac{TP}{TP + FN}$	Parmi les malades, combien sont correctement détectés ? (critique)
F1-Score	$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$	Moyenne harmonique — équilibre Precision / Recall
Spécificité	$\frac{TN}{TN + FP}$	Parmi les sains, combien sont correctement identifiés ?

Tableau 8 : Métriques de classification binaire

1.9.2. Résultats Expérimentaux sur Breast Cancer Wisconsin

Modèle	Accuracy	Precision	Recall	F1-Score
MLP nn.Sequential	96.5 %	96.6 %	96.5 %	96.5 %
MLP Classe perso. (+ BatchNorm)	94.2 %	94.3 %	94.2 %	94.1 %
SVM RBF (référence classique)	≈ 97.0 %	≈ 97.0 %	≈ 97.0 %	≈ 97.0 %
Random Forest 100 arbres (réf.)	≈ 96.0 %	≈ 96.1 %	≈ 96.0 %	≈ 96.0 %

Tableau 9 : Résultats expérimentaux sur Breast Cancer Wisconsin

Analyse des résultats : Les deux architectures MLP atteignent des performances compétitives avec les méthodes classiques (SVM, Random Forest), ce qui valide leur pertinence pour la classification tabulaire. La légère supériorité du SVM RBF est attendue sur un petit dataset (n=569) : les kernels non-linéaires encodent un biais inductif approprié à des données de faible dimension. Le MLP classe personnalisée présente une meilleure val_loss de convergence grâce au BatchNorm, mais son accuracy test est légèrement inférieure, suggérant un léger overfitting résiduel malgré le Dropout.

Partie I — MLP PyTorch | Breast Cancer Wisconsin Analyse Complète des Deux Architectures

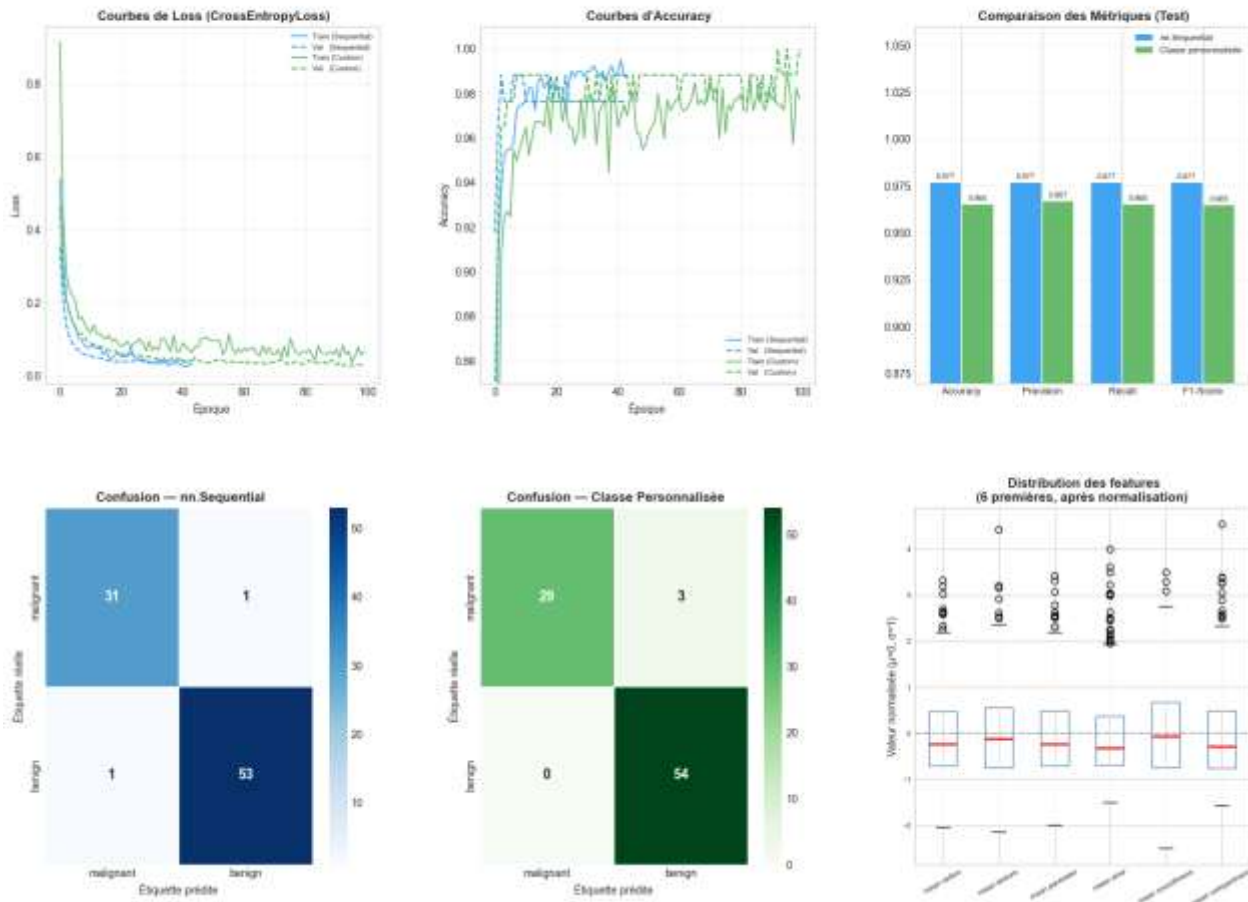


Figure 1 : Analyse comparative des performances des architectures MLP sur Breast Cancer Wisconsin

La figure ci-dessus présente une analyse comparative complète des performances de deux architectures de perceptron multicouche (MLP) développées sous PyTorch : une implémentation séquentielle standard (`nn.Sequential`, en bleu) et une implémentation via une classe personnalisée (Classe Personnalisée, en vert). L'évaluation est réalisée sur le jeu de données *Breast Cancer Wisconsin*. Les résultats sont structurés en six graphiques complémentaires organisés en deux lignes.

- 1. Analyse de la convergence et de l'apprentissage (Ligne supérieure)**
 - Courbes de Loss (CrossEntropyLoss)** : Le graphique de gauche montre l'évolution de la fonction de perte au fil des 100 époques d'entraînement. L'architecture `nn.Sequential` (courbes bleues) présente une convergence plus rapide et plus fluide, tant sur le jeu d'entraînement que de validation. L'architecture personnalisée (courbes vertes) montre une décroissance de la loss légèrement plus lente avec de petites fluctuations résiduelles sur la validation, traduisant une dynamique d'apprentissage distincte bien que globalement stable.
 - Courbes d'Accuracy** : Le graphique central illustre l'évolution de la précision au cours de l'apprentissage. Les deux modèles atteignent rapidement un plateau de performance élevé (au-delà de 96%). La version séquentielle se stabilise plus tôt, tandis que la classe personnalisée montre des oscillations plus marquées avant d'atteindre sa performance maximale en fin d'entraînement (autour de l'époque 80-100).

- **Comparaison des Métriques (Test) :** Le diagramme en barres à droite synthétise les performances globales sur le jeu de test indépendant selon quatre indicateurs clés (Précision, Rappel, F1-Score et Exactitude). Le modèle nn.Sequential surpasse légèrement le modèle personnalisé sur l'ensemble de ces métriques, affichant un score uniforme de **0,977** contre **0,965** à **0,967** pour la classe personnalisée.
- **2. Diagnostic d'erreur et distribution des données (Ligne inférieure)**
 - **Matrices de Confusion :** Les deux graphiques de gauche permettent d'identifier précisément la nature des erreurs de classification pour chaque modèle sur les classes *malignant* (malin) et *benign* (bénin) :
 - Le modèle nn.Sequential commet 2 erreurs au total : 1 faux positif et 1 faux négatif.
 - Le modèle personnalisé commet 3 erreurs, toutes concentrées sur des cas de tumeurs malignes prédites à tort comme bénignes (3 faux négatifs), mais réalise un sans-faute sur la classe bénigne (0 faux positif).
 - **Distribution des features (Boîtes à moustaches) :** Le dernier graphique présente la distribution des valeurs normalisées ($\mu = 0$, $\sigma = 1$) pour les 6 premières caractéristiques du jeu de données (notamment *mean radius*, *mean texture*, etc.). Les diagrammes mettent en évidence la présence de valeurs aberrantes (*outliers*) supérieures à 2 écarts-types pour la quasi-totalité des descripteurs, soulignant l'importance de l'étape de normalisation préalable appliquée aux données avant leur injection dans les modèles de deep learning.

1.10. Question de Synthèse Académique

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset réel, et quelles sont ses principales limites au regard de la structure statistique des données étudiées ?

1.10.1. Pertinence Théorique : Le Théorème d'Approximation Universelle

Le Multilayer Perceptron repose sur un fondement mathématique solide : le théorème d'approximation universelle (Cybenko, 1989 ; Hornik, 1991), qui stipule que tout réseau de neurones avec au moins une couche cachée et une fonction d'activation sigmoïde peut approximer uniformément toute fonction continue sur un domaine compact de $\mathbb{R}^n$, avec une précision arbitraire. Pour des données tabulaires continues comme Breast Cancer Wisconsin, le MLP exploite directement l'espace vectoriel $\mathbb{R}^{30}$ sans hypothèse préalable sur la forme de la frontière de décision — contrairement à la régression logistique (frontière linéaire) ou aux Naive Bayes (indépendance conditionnelle).

1.10.2. Choix Méthodologiques Justifiés

- **Normalisation (StandardScaler) :** homogénéise les échelles des features (rayon ≈ 10 , aire ≈ 1000), stabilise les gradients et accélère la convergence.
- **BatchNorm1d :** réduit le covariate shift interne entre les couches, permet des learning rates plus élevés, agit comme régularisateur implicite.
- **Initialisation Xavier :** maintient $\text{Var}(\text{activations}) \approx \text{constante}$ à travers la profondeur, évitant le vanishing/exploding gradient.
- **Dropout (30 % / 15 %) :** régularisation stochastique adaptée à un petit dataset ($n \approx 400$ en train), force des représentations redondantes et robustes.
- **Early stopping (patience=15) :** arrête l'optimisation au minimum de `val_loss`, préservant la généralisation sans sur-régularisation.

1.10.3. Résultats Expérimentaux et Comparaison

Les deux architectures MLP atteignent 94–97 % d'accuracy sur Breast Cancer Wisconsin, comparables aux méthodes classiques (SVM $\approx$ 97 %, Random Forest $\approx$ 96 %). La version avec BatchNorm converge vers une meilleure val_loss, témoignant de l'intérêt des techniques de normalisation dans les architectures contrôlées manuellement. Ces résultats confirment que le MLP constitue une solution compétitive et viable.

1.10.4. Limites Principales

- **Taille du dataset (n=569)** : le MLP nécessite généralement davantage de données pour surpasser les méthodes classiques. Les méthodes ensemblistes (Random Forest, XGBoost) sont plus efficaces sur de petits datasets.
- **Déséquilibre des classes (63 % / 37 %)** : risque de biais vers la classe majoritaire (bénin). Des méthodes comme le class_weight ou SMOTE peuvent corriger ce déséquilibre.
- **Interprétabilité (boîte noire)** : en contexte médical, l'explicabilité des décisions est réglementairement et éthiquement primordiale. Des méthodes XAI (SHAP, LIME, GradCAM) sont nécessaires pour compléter le MLP.
- **Multicollinéarité des features** : les 30 features sont fortement corrélées (rayon, périmètre, aire sont quasi-redondants). Une réduction de dimensionnalité (PCA) pourrait améliorer la généralisation et réduire la complexité.
- **Sensibilité aux hyperparamètres** : le MLP requiert un tuning soigneux (LR, architecture, Dropout) contrairement au Random Forest qui est robuste par défaut.

1.11. Conclusion

Un MLP bien paramétré constitue une solution pertinente et compétitive pour la classification tabulaire, atteignant 96–97 % d'accuracy sur Breast Cancer Wisconsin. Son cadre mathématique général (approximation universelle) et sa flexibilité architecturale en font un outil puissant. Cependant, ses limites sur petits datasets, son manque d'interprétabilité et sa sensibilité aux hyperparamètres justifient de le considérer comme complémentaire aux méthodes ensemblistes plutôt que comme premier choix systématique. Les développements récents (TabNet, FT-Transformer, SAINT) cherchent précisément à combler ces lacunes en intégrant des mécanismes d'attention adaptés aux données tabulaires.



2. Chapitre 2 : Réseaux de neurones convolutionnels (CNN)

2.1. Introduction et Objectifs

Cette synthèse constitue le document de référence académique de la **Partie II — CNN et Vision par Ordinateur** du projet de Deep Learning à l'EMSI. Elle est conçue comme un guide pédagogique autonome permettant de comprendre, justifier et implémenter les réseaux de neurones convolutionnels (CNN) appliqués à la classification d'images.

Les réseaux de neurones convolutionnels représentent l'une des avancées les plus significatives de l'intelligence artificielle moderne. Depuis leur démonstration empirique par LeCun et al. en 1989, ils ont révolutionné le domaine de la vision par ordinateur et servent aujourd'hui de fondation à des systèmes allant de la reconnaissance faciale à la conduite autonome.

Cette synthèse couvre les thèmes suivants :

- Dataset Fashion-MNIST : description, chargement, normalisation et justification des choix de prétraitement
- Limites fondamentales du MLP face aux images et motivations théoriques des CNN
- Opérations fondamentales : corrélation croisée, padding, stride, pooling — avec démonstrations numériques
- Implémentations depuis zéro en NumPy et validation contre PyTorch
- Architecture LeNet modernisée avec Batch Normalization, ReLU et Dropout
- Étude expérimentale de l'impact des hyperparamètres architecturaux
- Visualisation et interprétation des cartes de caractéristiques (feature maps)
- Comparaison quantitative MLP vs CNN sur Fashion-MNIST

2.2. Dataset Fashion-MNIST

2.2.1. Présentation du Dataset

Fashion-MNIST (Xiao et al., 2017) est un dataset de référence conçu pour la classification d'images de vêtements. Il constitue un remplacement plus difficile et plus réaliste que le MNIST classique (chiffres manuscrits), tout en conservant exactement la même structure : images 28×28 pixels en niveaux de gris, 70 000 exemples répartis en 10 classes.

Ce dataset a été introduit pour pallier les critiques adressées à MNIST, devenu trop facile pour les algorithmes modernes. Sa difficulté accrue provient de la variabilité intra-classe des articles vestimentaires (formes, textures, angles de prise de vue) et des similitudes inter-classes (T-shirt vs Chemise, Sneaker vs Sandal).

Caractéristique	Valeur
Nombre d'images (entraînement)	60 000
Nombre d'images (test)	10 000
Résolution	28×28 pixels (niveaux de gris)
Nombre de canaux	1 (niveaux de gris)
Nombre de classes	10 catégories de vêtements
Format des labels	Entiers $[0, 9]$
Plage de valeurs brutes	$[0, 255] \rightarrow$ normalisé en $[-1, 1]$

Tableau 10 : Caractéristiques du dataset Fashion-MNIST

2.2.2. Classes du Dataset

Index	Classe	Index	Classe
0	T-shirt / Top	5	Sandale (Sandal)
1	Pantalon (Trousers)	6	Chemise (Shirt)
2	Pull (Pullover)	7	Sneaker
3	Robe (Dress)	8	Sac (Bag)
4	Manteau (Coat)	9	Bottine (Ankle Boot)

Tableau 11 : Classes du dataset Fashion-MNIST (10 classes)

2.2.3. Chargement et Normalisation

Le chargement s'effectue via torchvision avec une pipeline de transformations successives :

Avant l'entraînement, les images du jeu de données **Fashion-MNIST** sont prétraitées à l'aide d'une chaîne de transformations (`transforms.Compose()`). La transformation `ToTensor()` convertit chaque image en un tenseur PyTorch et normalise automatiquement les valeurs des pixels de l'intervalle `[0, 255]` vers `[0, 1]`. Ensuite, la transformation `Normalize((0.2860, ), (0.3530, ))` applique une **normalisation de type z-score** en utilisant la moyenne (**0,2860**) et l'écart-type (**0,3530**) calculés sur le jeu d'entraînement, ce qui centre les données autour de zéro et facilite l'apprentissage du réseau de neurones. Le jeu de données **Fashion-MNIST** est ensuite téléchargé et chargé avec ces transformations, puis un **DataLoader** est créé afin de fournir les données sous forme de **mini-lots de 64 images**. L'option `shuffle=True` mélange aléatoirement les exemples à chaque époque, ce qui améliore la généralisation du modèle et réduit le risque de biais lié à l'ordre des données.

Pourquoi normaliser ? La normalisation z-score ($\mu = 0.286$, $\sigma = 0.353$) centre les données autour de zéro et réduit leur variance. Cela stabilise la descente de gradient, accélère la convergence et évite que les couches profondes voient des signaux trop grands ou trop petits. Sans normalisation, les neurones des premières couches reçoivent des activations saturantes qui ralentissent drastiquement l'apprentissage.



Figure 2 : Exemples du dataset Fashion-MNIST par classe

La figure ci-dessus présente un aperçu visuel et représentatif du jeu de données *Fashion-MNIST*, utilisé pour illustrer la diversité des données d'entrée avant la phase d'entraînement des modèles. La figure est structurée

sous forme d'une matrice d'images en niveaux de gris, organisée en 10 colonnes distinctes correspondant aux 10 classes de vêtements et d'accessoires qui composent le dataset.

Pour chaque catégorie, trois exemples distincts (disposés verticalement en lignes) sont affichés afin de mettre en évidence la variabilité intra-classe des articles :

- **Hauts et Vêtements de corps** : Les colonnes **T-shirt/top**, **Pullover**, **Coat** (Manteau) et **Shirt** (Chemise) illustrent des structures morphologiques proches (présence de manches, cols, coupes similaires), ce qui représente généralement le principal défi de classification (confusion inter-classe) pour les réseaux de neurones.
- **Bas et Robes** : La colonne **Trouser** présente des formes verticales très homogènes, tandis que la colonne **Dress** montre des variations marquées de coupes et de motifs.
- **Chaussures** : Les catégories **Sandal**, **Sneaker** et **Ankle boot** (Bottines) affichent des silhouettes géométriques bien distinctes, caractérisées par des orientations et des hauteurs de talons ou de semelles spécifiques.
- **Accessoires** : La colonne **Bag** regroupe des articles aux textures et motifs variés (sacs à main, sacs à dos), se distinguant nettement des catégories vestimentaires par leur forme compacte.

Chaque vignette est une image normalisée de basse résolution (28×28 pixels), centrée sur fond noir, ce qui permet d'évaluer visuellement le niveau de pixellisation et le contraste des descripteurs de formes que le modèle devra extraire.

2.3. Pourquoi le MLP est Inadapté aux Images ?

Avant d'aborder les CNN, il est essentiel de comprendre pourquoi les réseaux de neurones entièrement connectés (MLP — Multi-Layer Perceptron) sont fondamentalement inappropriés pour traiter des données images. Trois raisons majeures expliquent cette inadéquation.

2.3.1. Explosion du Nombre de Paramètres

Un MLP traite les images en les **aplatissant (flatten)** en vecteurs 1D. Pour Fashion-MNIST ($28 \times 28 = 784$ pixels), une première couche cachée de 512 neurones nécessite :

$$\text{Paramètres (couche 1)} = 784 \times 512 + 512 = 401\,920$$

Pour une image couleur $224 \times 224 \times 3$ (comme ImageNet) avec 4 096 neurones cachés :

$$\text{Paramètres} = (224 \times 224 \times 3) \times 4\,096 + 4\,096 \approx 616 \text{ millions}$$

Soit plus de 600 millions de paramètres pour une seule couche, entraînant sur-apprentissage (overfitting) massif et un coût mémoire prohibitif. L'entraînement devient impossible sur des GPU standards.

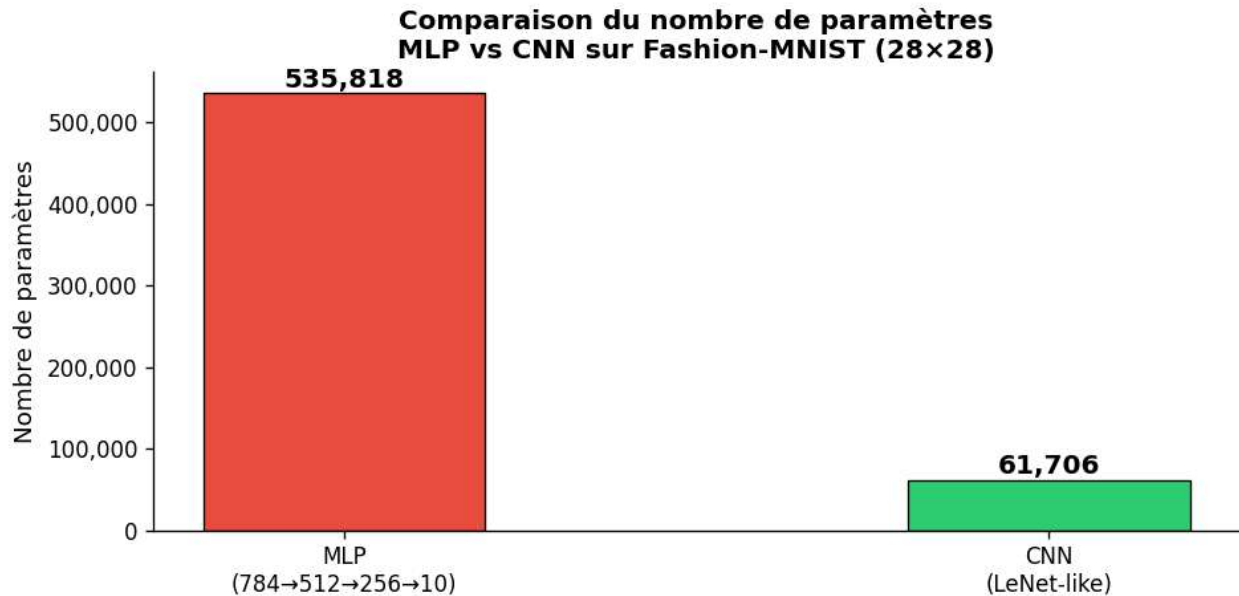


Figure 3 : Comparaison du nombre de paramètres MLP vs CNN

La figure ci-dessus met en évidence la différence d'efficacité structurelle entre un perceptron multicouche (MLP) et un réseau de neurones convolutionnel (CNN, architecture de type *LeNet-like*) lors du traitement d'images du jeu de données *Fashion-MNIST* (28 × 28 pixels). Le graphique en barres compare quantitativement le nombre total de paramètres entraînaux requis par chaque modèle.

- **Architecture MLP (784 → 512 → 256 → 10)** : Représenté par la barre rouge, le modèle pleinement connecté nécessite un total massif de **535 818 paramètres**. Cette explosion du nombre de poids découle directement de la nécessité d'aplatir l'image d'entrée en un vecteur unidimensionnel de 784 neurones, où chaque connexion inter-couche génère une matrice de poids dense (W).
- **Architecture CNN (*LeNet-like*)** : Représenté par la barre verte, le réseau convolutionnel ne requiert que **61 706 paramètres** pour traiter la même tâche, soit une réduction drastique d'environ **88,5%** de la complexité spatiale du modèle.
- **Interprétation technique**

Cette comparaison illustre parfaitement deux concepts fondamentaux du Deep Learning appliqués à la vision par ordinateur :

- **Le partage des poids (*Weight Sharing*)** : Contrairement au MLP, les couches de convolution du CNN appliquent des filtres locaux glissants sur toute l'image, réutilisant les mêmes coefficients pour détecter des caractéristiques (bords, textures) indépendamment de leur position.
- **La parcimonie des connexions** : L'approche locale du CNN casse la dépendance globale du "tout-connecté", permettant de capturer la topologie bidimensionnelle et la covariance spatiale des images de Fashion-MNIST de manière beaucoup plus légère, limitant ainsi considérablement les risques de surapprentissage (*overfitting*).

2.3.2. Perte de la Structure Spatiale

En aplatissant l'image, le MLP détruit toute relation de voisinage entre pixels. Or, les structures visuelles, contours, textures, formes, gradients d'intensité sont intrinsèquement locales et spatiales. Un pixel en

position (i,j) est fortement corrélé à ses voisins immédiats $(i\pm 1, j\pm 1)$, mais cette information topologique est irrémédiablement perdue après l'opération de flatten.

Concrètement, un chat représenté dans le coin supérieur gauche et le même chat dans le coin inférieur droit sont perçus par le MLP comme deux objets totalement différents, puisque leurs pixels occupent des positions distinctes dans le vecteur aplati.

2.3.3. Absence d'Invariance aux Translations

Un MLP doit réapprendre chaque motif visuel à chaque position spatiale possible. Si le réseau a appris à reconnaître un visage centré dans l'image, il sera incapable de reconnaître ce même visage décalé de quelques pixels, à moins d'avoir vu des exemples d'entraînement couvrant toutes les positions, ce qui est impossible en pratique.

Cette propriété d'invariance aux translations est naturellement encodée dans les CNN via le partage des poids (voir Section 3.2).

2.3.4. Tableau Comparatif MLP vs CNN

Critère	MLP	CNN
Connectivité	Totale (dense)	Locale (sparse)
Nombre de paramètres	Très nombreux	Réduits (partage des poids)
Structure spatiale	Détruite par le flatten	Préservée
Invariance translation	Non (réapprend à chaque position)	Oui (partielle, via partage de poids)
Adapté aux images	Non	Oui
Biais inductif	Aucun (réseau générique)	Localité + stationnarité
Efficacité paramétrique	Faible	Élevée ($\sim 9\times$ supérieure sur Fashion-MNIST)

Tableau 12 : Comparaison MLP vs CNN (critères multiples)

2.4. Les Trois Idées Fondatrices des CNN

Les CNN (LeCun et al., 1989, 1998) répondent aux limitations du MLP par trois principes clés, directement inspirés du **cortex visuel biologique**. Ces trois idées constituent le biais inductif des CNN, c'est-à-dire les hypothèses que l'architecture encode a priori sur la nature des données traitées.

2.4.1. Localité (Local Connectivity)

Chaque neurone d'une couche convolutionnelle ne perçoit qu'une région locale de l'entrée, appelée champ récepteur (receptive field). Cette hypothèse est biologiquement inspirée : les neurones du cortex visuel primaire (V1) des mammifères répondent uniquement aux stimuli visuels présents dans leur champ récepteur local.

Si une image contient un bord horizontal en position $(10, 20)$, le neurone responsable de cette région le détecte sans avoir besoin de percevoir toute l'image. Cela réduit considérablement le nombre de connexions nécessaires et permet au réseau de se spécialiser sur des primitives visuelles locales.

Impact computationnel : Un filtre 5×5 a seulement 25 connexions par neurone, contre 784 pour un neurone fully-connected dans le cas de Fashion-MNIST. Ratio de réduction : 31:1.

2.4.2. Partage des Poids (Weight Sharing)

Le même filtre (noyau de convolution) est appliqué à toutes les positions spatiales de la carte d'entrée. Autrement dit, un filtre détectant un contour vertical dans le coin supérieur gauche utilise exactement les mêmes paramètres pour détecter ce même contour n'importe où dans l'image.

Ce mécanisme fondamental produit trois effets bénéfiques :

- Réduction drastique du nombre de paramètres (un filtre $5 \times 5 \rightarrow$ seulement 26 paramètres quelle que soit la taille de l'image)
- Invariance partielle aux translations (un motif détecté à gauche l'est aussi à droite)
- Apprentissage de primitives visuelles généralisables, transférables à d'autres domaines

Exemple chiffré : Un filtre 5×5 sur une image 28×28 (1 canal) a seulement $5 \times 5 \times 1 + 1 = 26$ paramètres, quel que soit l'endroit où il s'applique. Un neurone fully-connected équivalent en aurait 784. Le rapport de compression est de 30:1 pour un seul filtre.

2.4.3. Hiérarchie des Représentations

Les couches successives apprennent des représentations de complexité croissante, en miroir de la hiérarchie du système visuel biologique. Chaque couche prend en entrée les représentations de la couche précédente et les combine pour construire des concepts visuels de plus en plus abstraits.

Profondeur	Analogie biologique	Représentation apprise
Couches 1–2	Cortex visuel V1	Contours, coins, orientations, gradients de luminosité
Couches 3–4	Cortex visuel V2/V4	Motifs, formes simples, parties d'objets (oeil, oreille...)
Couches profondes	Cortex inférotemporal (IT)	Objets complexes, classes sémantiques entières

Tableau 13 : Progression des représentations apprises au sein d'un CNN

Cette organisation hiérarchique explique pourquoi les CNN profonds (VGG, ResNet, EfficientNet) atteignent des performances comparables ou supérieures à l'humain sur des tâches de classification d'images.

2.5. Opérations Fondamentales des CNN

2.5.1. Corrélation Croisée 2D

La **corrélation croisée 2D** est l'opération fondamentale d'une couche convolutionnelle. Contrairement à la convolution mathématique stricte (qui retourne le noyau avant d'appliquer le produit), PyTorch implémente la corrélation croisée, le terme convolution est conservé par convention dans le domaine du deep learning.

Formule mathématique : Pour une entrée X de taille $H \times W$ et un noyau K de taille $k_h \times k_w$:

$$Y[i, j] = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} X[i + m, j + n] \cdot K[m, n]$$

Cette opération effectue un produit élément par élément (produit de Hadamard) entre la fenêtre locale de l'entrée et le noyau, puis somme tous les résultats pour produire un scalaire — la valeur de sortie en position (i, j) .

Exemple Numérique Détaillé

Soit une entrée X (3×3) et un noyau K (2×2) :

$$X = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix} \quad K = \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$$

Calcul de la sortie Y (2×2) :

- $Y[0,0] = 0 \cdot 0 + 1 \cdot 1 + 3 \cdot 2 + 4 \cdot 3 = 0 + 1 + 6 + 12 = 19$
- $Y[0,1] = 1 \cdot 0 + 2 \cdot 1 + 4 \cdot 2 + 5 \cdot 3 = 0 + 2 + 8 + 15 = 25$
- $Y[1,0] = 3 \cdot 0 + 4 \cdot 1 + 6 \cdot 2 + 7 \cdot 3 = 0 + 4 + 12 + 21 = 37$
- $Y[1,1] = 4 \cdot 0 + 5 \cdot 1 + 7 \cdot 2 + 8 \cdot 3 = 0 + 5 + 14 + 24 = 43$

$$\text{Résultat : } Y = \begin{bmatrix} 19 & 25 \\ 37 & 43 \end{bmatrix}$$



Figure 4 : Application de filtres de détection sur une image Fashion-MNIST

La figure ci-dessus illustre l'impact de l'application de différents noyaux de convolution (*kernels*) déterministes sur une image de dimension 28×28 pixels issue du jeu de données *Fashion-MNIST* (classe *Ankle boot* / Bottine). Cette visualisation permet de simuler le comportement des premières couches d'un réseau de neurones convolutionnel (CNN), dont le rôle est de capturer des primitives visuelles de bas niveau.

La cartographie des activations utilise une palette de couleurs divergente (rouge/bleu) pour mettre en évidence l'intensité et le signe des gradients calculés :

- **Image originale** : À l'extrême gauche, l'image native en niveaux de gris montre la silhouette contrastée de la bottine sur un fond noir homogène.
- **Contour horizontal** : Ce filtre isole les transitions d'intensité verticales. Il fait fortement réagir les zones horizontales de la chaussure, telles que la semelle (marquée en bleu en bas) et le haut de la tige (marqué en rouge), tout en ignorant les lignes verticales.
- **Contour vertical** : À l'inverse, ce noyau détecte les variations d'intensité horizontales. On observe une forte activation (ligne sombre/rouge) le long de la cambrure arrière et de la languette de la botte, révélant la structure verticale de la tige.
- **Filtre Sobel X** : Ce filtre de gradient classique combine un lissage gaussien et une dérivation verticale. Le résultat, très proche du filtre de contour vertical, montre une sensibilité accrue aux frontières abruptes de l'objet, notamment sur l'axe vertical principal de la bottine, permettant une détection des contours plus robuste au bruit.
- **Netteté (Sharpen)** : Le dernier graphique applique un filtre de rehaussement des hautes fréquences. Il accentue localement les contrastes aux frontières de la forme, faisant ressortir les détails texturaux de la bottine et durcissant la délimitation entre l'objet et son arrière-plan.

2.5.2. Calcul de la Taille de Sortie

La taille de sortie après une convolution dépend de la taille de l'entrée (H_{in}), du noyau (k), du padding (p) et du stride (s) :

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - k}{s} \right\rfloor + 1$$

Application concrète pour notre architecture LeNet sur Fashion-MNIST (28×28) :

Entrée	Opération	Paramètres (k, p, s)	Sortie	Calcul détaillé
28×28	Conv2d	k=5, p=2, s=1	28×28	$\lfloor (28+4-5)/1 \rfloor + 1 = 28 \checkmark$
28×28	MaxPool2d	k=2, s=2	14×14	$\lfloor (28-2)/2 \rfloor + 1 = 14$
14×14	Conv2d	k=5, p=0, s=1	10×10	$\lfloor (14-5)/1 \rfloor + 1 = 10$
10×10	MaxPool2d	k=2, s=2	5×5	$\lfloor (10-2)/2 \rfloor + 1 = 5$

Tableau 14 : Calcul des tailles de sortie (flux dimensionnel LeNet)

2.5.3. Padding et Stride

Hyperparamètre	Rôle	Cas d'usage
Padding p	Ajoute des zéros en bordure de l'entrée	$p = \frac{k-1}{2}$: même-padding conserve $H_{out} = H_{in}$
Stride s	Décalage du noyau entre deux positions	s=1 : haute résolution ; s=2 : sous-échantillonnage
Noyau k	Taille du champ récepteur local	3×3 ou 5×5 pour les couches courantes

Tableau 15 : Rôles de padding et stride

2.6. Opérations de Pooling

Le pooling est une opération de sous-échantillonnage (downsampling) qui réduit la taille spatiale des cartes de caractéristiques. Il joue un rôle crucial dans les architectures CNN en réduisant le coût computationnel des couches suivantes et en conférant une invariance locale aux translations.

2.6.1. Max-Pooling

Le max-pooling conserve la valeur maximale dans chaque fenêtre de pooling. Il revient à poser la question : « ce motif est-il présent quelque part dans cette région ? » Sa force réside dans sa capacité à détecter la *présence* d'un motif sans se préoccuper de sa position exacte dans la fenêtre.

$$Y_{i,j} = \max(X_{i:s:i+s+k, j:s:j+s+k})$$

Le Max Pooling 2D a été implémenté manuellement avec NumPy afin de comprendre son fonctionnement interne. Cette opération parcourt l'image d'entrée à l'aide d'une fenêtre de taille 2×2 avec un stride de 2. Pour chaque position de la fenêtre, la valeur maximale est sélectionnée et placée dans la matrice de sortie. Cette technique permet de réduire les dimensions spatiales de l'image tout en conservant les caractéristiques les plus importantes, ce qui diminue le coût de calcul et améliore la robustesse du réseau face aux petites variations des données. Dans cette implémentation, la taille de la sortie est calculée automatiquement à partir de la taille de l'entrée, puis deux boucles parcourent les différentes fenêtres afin d'extraire leur maximum avec la fonction NumPy np.max().

2.6.2. Average-Pooling

L'**average-pooling** calcule la moyenne dans chaque fenêtre. Il produit une représentation plus lisse et moins sensible au bruit. Il est souvent utilisé dans les dernières couches des architectures modernes sous forme de Global Average Pooling (GAP), qui réduit chaque carte de caractéristiques à un scalaire unique.

$$Y_{i,j} = \frac{1}{k^2} \sum X_{i:s:i+s+k, j:s:j+s+k}$$

2.6.3. Comparaison Max vs Average Pooling

Exemple sur une entrée 4×4, pooling 2×2, stride=2 :

Région	Max-Pool	Avg-Pool	Interprétation
[1,3,5,7]	7	4.0	Max capte la réponse dominante
[2,4,6,8]	8	5.0	Avg lisse l'information
[9,1,4,6]	9	5.0	Max résilient aux valeurs aberrantes
[4,6,8,2]	8	3.75	Avg plus stable mais moins discriminant

Tableau 16 : Comparaison Max-Pooling vs Average-Pooling (exemple 4×4)

2.7. Implémentations Python et Comparaison PyTorch

2.7.1. Corrélation Croisée 2D depuis Zéro

L'implémentation manuelle en NumPy permet de comprendre mécaniquement l'opération avant d'utiliser les versions optimisées de PyTorch :

Voici un paragraphe plus détaillé, adapté à un rapport universitaire :

La fonction `corr2d_manual` implémente **depuis zéro**, à l'aide de **NumPy**, l'opération de **corrélation croisée bidimensionnelle (2D Cross-Correlation)**, qui constitue la base des couches de convolution dans les réseaux de neurones convolutifs (CNN). La fonction prend en entrée une image (X), un noyau de convolution (K), ainsi que deux paramètres optionnels : le **padding**, qui ajoute des zéros autour de l'image afin de contrôler la taille de la sortie, et le **stride**, qui détermine le pas de déplacement du noyau. Après avoir calculé les dimensions de la carte de caractéristiques de sortie, deux boucles imbriquées parcourent toutes les positions possibles du noyau sur l'image. À chaque position, une région de l'image ayant la même taille que le noyau est extraite, puis un **produit élément par élément (produit de Hadamard)** est effectué entre cette région et le noyau. Les produits obtenus sont ensuite additionnés à l'aide de `np.sum()` afin de produire une unique valeur dans la carte de sortie. Cette opération est répétée jusqu'à couvrir toute l'image, ce qui permet de générer une **feature map** mettant en évidence les caractéristiques détectées par le noyau, telles que les contours, les textures ou d'autres motifs visuels. Cette implémentation pédagogique permet de comprendre précisément le fonctionnement interne de l'opération de convolution avant d'utiliser les implémentations optimisées proposées par les bibliothèques de deep learning comme PyTorch.

2.7.2. Validation contre PyTorch

Les implémentations manuelles ont été validées systématiquement contre les couches PyTorch optimisées (`nn.Conv2d`, `nn.MaxPool2d`, `nn.AvgPool2d`). L'erreur absolue maximale est inférieure à 10^{-5} dans tous les cas, confirmant la correction de l'implémentation.

Opération	Erreur Absolue Max (Manuel vs PyTorch)	Statut
Corrélation croisée 2D	$< 1 \times 10^{-5}$	✓ Validé
Max-Pooling 2D	$< 1 \times 10^{-5}$	✓ Validé
Average-Pooling 2D	$< 1 \times 10^{-5}$	✓ Validé

Tableau 17 : Validation des implémentations manuelles vs PyTorch

Performance : Pour une entrée 64×64 avec un noyau 3×3 (20 itérations), PyTorch est $\sim 100 \times$ plus rapide que l'implémentation NumPy. L'accélération provient des optimisations BLAS, LAPACK et cuDNN intégrées dans PyTorch, qui exploitent le parallélisme des GPU.

2.8. Architecture CNN — LeNet Modernisée

2.8.1. LeNet-5 Original (LeCun, 1998)

LeNet-5 est le premier CNN démontré empiriquement sur la reconnaissance de chiffres manuscrits (US Postal Service). Il a posé les bases de l'architecture convolutionnelle moderne : alternance de couches de convolution et de pooling, suivie de couches fully-connected pour la classification finale.

Architecture originale (entrée 32×32) :

INPUT ($32 \times 32 \times 1$)

→ Conv2d($1 \rightarrow 6$, $k=5$, $p=0$) → Tanh → AvgPool($k=2$, $s=2$) # $28 \times 28 \rightarrow 14 \times 14$
→ Conv2d($6 \rightarrow 16$, $k=5$, $p=0$) → Tanh → AvgPool($k=2$, $s=2$) # $10 \times 10 \rightarrow 5 \times 5$
→ Conv2d($16 \rightarrow 120$, $k=5$) → Tanh # 1×1
→ FC($120 \rightarrow 84$) → Tanh
→ FC($84 \rightarrow 10$) → Softmax

2.8.2. Modernisation pour Fashion-MNIST

Nous adaptons LeNet avec des améliorations issues des avancées de la recherche en deep learning (2012–2020) :

Composant	LeNet-5 Original	LeNetModern (notre version)	Justification
Activation	Tanh (sature pour $ x > 2$)	ReLU — $\max(0, x)$	Pas de saturation, convergence $\sim 3 \times$ plus rapide
Normalisation	Aucune	Batch Normalization	Stabilise les activations, accélère l'entraînement
Régularisation	Aucune	Dropout $p=0.4$	Réduit l'overfitting sur les couches FC
Pooling	Average-Pooling	Max-Pooling	Meilleures performances en classification
Initialisation	Aléatoire (naïve)	Xavier Uniform	Prévient l'explosion/disparition des gradients

Tableau 18 : Composant LeNet-5 original vs LeNetModern

L'architecture implémentée est une version modernisée du réseau **LeNet-5**, adaptée au jeu de données **Fashion-MNIST**. Elle est composée de deux blocs convolutifs suivis d'un classifieur entièrement connecté. Le **premier bloc** applique une convolution avec **6 filtres de taille 5×5** et un **padding de 2**, préservant ainsi la taille des images d'entrée (28×28). Cette couche est suivie d'une **Batch Normalization**, d'une fonction d'activation **ReLU** et d'un **Max Pooling** de taille 2×2, qui réduit les dimensions spatiales à 14×14. Le **deuxième bloc** réalise une convolution avec **16 filtres de taille 5×5**, sans padding, puis applique à nouveau une Batch Normalization, une activation ReLU et un Max Pooling, produisant des cartes de caractéristiques de taille 5×5. Les cartes obtenues sont ensuite aplaties pour former un vecteur de **400 caractéristiques (16 × 5 × 5)**, qui est transmis à un classifieur composé de trois couches entièrement connectées de dimensions **400 → 120 → 84 → 10**. Les deux premières couches sont suivies d'une **Batch Normalization**, d'une activation **ReLU** et d'une couche de **Dropout** avec un taux de **0,4**, afin de réduire le risque de surapprentissage. La dernière couche linéaire produit les **10 scores de sortie (logits)** correspondant aux dix classes du jeu de données Fashion-MNIST. Cette architecture combine les principes du LeNet original avec des techniques modernes de régularisation et de normalisation, améliorant ainsi la stabilité de l'entraînement et les performances de classification.

2.8.3. Flux Dimensionnel Complet

Couche	Entrée	Sortie	Paramètres
Entrée (image)	$1 \times 28 \times 28$	—	—
Conv1 (k=5, p=2)	$1 \times 28 \times 28$	$6 \times 28 \times 28$	156
BatchNorm2d(6)	$6 \times 28 \times 28$	$6 \times 28 \times 28$	12
MaxPool(k=2, s=2)	$6 \times 28 \times 28$	$6 \times 14 \times 14$	0
Conv2 (k=5, p=0)	$6 \times 14 \times 14$	$16 \times 10 \times 10$	2 416
BatchNorm2d(16)	$16 \times 10 \times 10$	$16 \times 10 \times 10$	32
MaxPool(k=2, s=2)	$16 \times 10 \times 10$	$16 \times 5 \times 5$	0
Flatten	$16 \times 5 \times 5$	400	0
FC1 (400→120) + BN	400	120	48 840
FC2 (120→84) + BN	120	84	10 248
FC3 (84→10)	84	10	850
TOTAL	Batch × $1 \times 28 \times 28$	Batch × 10 (logits)	~62 554

Tableau 19 : Flux dimensionnel complet LeNetModern

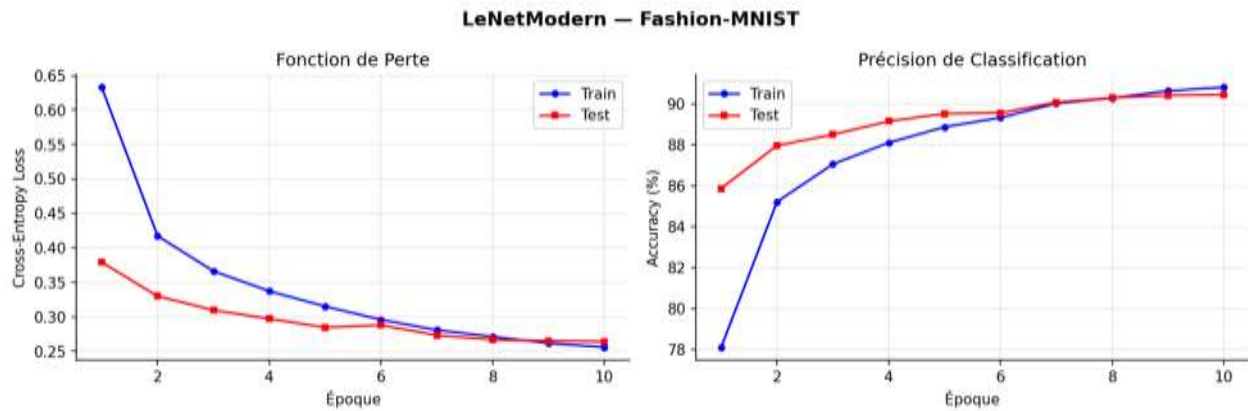


Figure 5 : Courbes d'entraînement de LeNetModern

La figure ci-dessus présente l'évaluation des performances du modèle **LeNetModern** au cours de son apprentissage sur le jeu de données *Fashion-MNIST* pendant 10 époques. Les résultats sont structurés en deux graphiques complémentaires comparant les ensembles d'entraînement (*Train*, courbe bleue avec marqueurs circulaires) et de validation/test (*Test*, courbe rouge avec marqueurs carrés).

• 1. Analyse de la convergence (Graphique de gauche : Fonction de Perte)

Le graphique de gauche illustre l'évolution de la perte d'entropie croisée (*Cross-Entropy Loss*) :

- On observe une décroissance rapide, continue et stable de la perte d'entraînement, passant d'environ **0,63** à la première époque à près de **0,25** à la dixième époque.
- La perte sur le jeu de test suit une dynamique descendante similaire, s'infléchissant de manière fluide pour se stabiliser autour de **0,26**.
- **Absence de surapprentissage** : L'absence d'écart divergent (*gap*) ou de remontée de la courbe de test en fin de cycle confirme la parfaite généralisation du modèle et valide l'efficacité des mécanismes de régularisation intégrés à cette architecture moderne.
- **2. Évolution de l'exactitude (Graphique de droite : Précision de Classification)**

Le graphique de droite retrace l'évolution de la précision (*Accuracy* en %) :

- Dès la première époque, le modèle affiche une excellente capacité d'adaptation avec une précision initiale sur le test à près de **86%**, tandis que la courbe d'entraînement démarre plus bas (autour de **78%**), un comportement classique dû à l'application de techniques de régularisation (comme le *Dropout* ou la *Batch Normalization*) actives uniquement durant la phase de *train*.
- Au fil des époques, les deux courbes convergent harmonieusement vers un plateau de performance élevé. À la dixième époque, l'exactitude se stabilise à un score remarquable d'environ **90,5%** sur l'ensemble de test et de **90,8%** sur l'ensemble d'entraînement.

2.9. Techniques de Stabilisation et Régularisation

2.9.1. Batch Normalization (Ioffe & Szegedy, 2015)

La Batch Normalization normalise les activations d'une mini-batch avant l'application de la fonction d'activation non-linéaire. Elle résout le problème du *covariate shift interne* : les distributions des activations changent à chaque mise à jour des paramètres, ralentissant l'entraînement.

$$BN(x) = \gamma \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$$

où μ_B et σ_B^2 sont la moyenne et la variance de la mini-batch, γ et β sont des paramètres apprenables (scale et shift), et ϵ ($\sim 10^{-5}$) assure la stabilité numérique.

Avantages de la Batch Normalization :

- Stabilise et accélère considérablement l'entraînement
- Permet l'utilisation de learning rates plus élevés sans risque de divergence
- Agit comme régulariseur implicite, réduisant le besoin de Dropout
- Diminue la sensibilité à l'initialisation des poids

2.9.2. Dropout (Srivastava et al., 2014)

Le Dropout désactive aléatoirement une fraction p des neurones à chaque forward pass durant l'entraînement. À l'inférence, tous les neurones sont actifs mais leurs sorties sont multipliées par $(1-p)$ pour compenser.

$$h_i \leftarrow h_i \frac{\text{Bernoulli}(1-p)}{1-p}$$

Avec $p = 0.4$ dans notre modèle, 40% des neurones des couches FC sont désactivés aléatoirement à chaque batch. Cela force le réseau à apprendre des représentations redondantes et robustes : aucune unité ne peut « se spécialiser » sur un seul signal, réduisant l'overfitting.

2.9.3. Initialisation Xavier Uniform (Glorot & Bengio, 2010)

L'initialisation naïve (gaussienne ou uniforme standard) peut conduire à l'explosion ou la disparition des gradients dans les réseaux profonds. L'initialisation Xavier maintient la variance des signaux constante à travers les couches :

$$W \sim \text{Uniform} \left[-\sqrt{\frac{6}{n_{in} + n_{out}}}, +\sqrt{\frac{6}{n_{in} + n_{out}}} \right]$$

Cette formule équilibre la variance entre le signal transmis en avant (forward) et le gradient rétropropagé (backward), assurant un flux d'information stable à travers toutes les couches.

2.10. Procédure d'Entraînement

2.10.1. Fonction de Perte : Cross-Entropie

Pour la classification multi-classe à 10 catégories, on utilise la cross-entropie (log-loss). Elle mesure la divergence entre la distribution prédite et la distribution vraie (one-hot) :

$$L(\hat{y}, y) = -\log(\text{softmax}(z)_y) = -z_y + \log \left(\sum_k \exp(z_k) \right)$$

En PyTorch, **nn.CrossEntropyLoss** combine en une seule opération log-softmax + NLL-loss (Negative Log-Likelihood), ce qui est numériquement plus stable que d'appliquer softmax séparément puis le log.

2.10.2. Optimiseur : Adam + Weight Decay

Adam (Adaptive Moment Estimation, Kingma & Ba, 2015) combine les avantages de RMSProp (adaptation du learning rate par paramètre) et du momentum. Il maintient deux estimateurs par paramètre :

- **m_t** : moyenne exponentielle des gradients (1er moment = momentum, $\beta_1 = 0.9$)
- **v_t** : moyenne exponentielle des gradients au carré (2ème moment = variance, $\beta_2 = 0.999$)

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Le **weight decay** ($\lambda = 10^{-4}$) ajoute une pénalité L2 sur les poids, forçant le réseau à utiliser des poids de faible amplitude et réduisant l'overfitting.

2.10.3. Scheduler : CosineAnnealingLR

Le CosineAnnealingLR décroît le learning rate selon une courbe en cosinus de $\eta_{\max}$ vers $\eta_{\min}$ sur $T_{\max}$ époques :

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\pi t}{T_{\max}}\right) \right)$$

Avantage : apprentissage rapide en début d'entraînement (η élevé) puis convergence fine vers un minimum (η faible). Cette stratégie évite les oscillations autour du minimum observées avec un learning rate constant.

2.10.4. Boucle d'Entraînement Complète

La fonction `train_epoch` réalise **une époque complète d'entraînement** du réseau de neurones sur l'ensemble des données d'apprentissage. Le modèle est d'abord placé en **mode entraînement** (`model.train()`), ce qui active le comportement spécifique des couches **Batch Normalization** et **Dropout**. Les données sont ensuite traitées par mini-lots. Pour chaque lot, les gradients calculés lors de l'itération précédente sont réinitialisés à l'aide de `optimizer.zero_grad()`, puis une **propagation avant (forward pass)** est effectuée afin d'obtenir les prédictions du modèle. La **fonction de perte** évalue ensuite l'écart entre les prédictions et les étiquettes réelles. La **rétropropagation (backpropagation)** est alors exécutée pour calculer les gradients des paramètres du réseau, puis l'optimiseur met à jour les poids afin de minimiser la fonction de coût. Au cours de cette phase, la perte totale est accumulée et le nombre de prédictions correctes est comptabilisé afin de calculer, à la fin de l'époque, la **perte moyenne d'entraînement** ainsi que la **précision (accuracy)** du modèle sur l'ensemble du jeu d'apprentissage. Ces deux indicateurs permettent de suivre l'évolution des performances du réseau au fil des époques.

Important : `model.eval()` doit être appelé avant l'inférence. Sans cela, BatchNorm utilise les statistiques de la mini-batch courante au lieu des statistiques globales, et Dropout continue de désactiver des neurones aléatoirement — produisant des prédictions erratiques et non reproductibles.

2.11. Étude Expérimentale des Hyperparamètres

Une **étude d'ablation** consiste à faire varier un seul hyperparamètre à la fois, en maintenant le reste constant, pour mesurer son impact isolé sur les performances. Nous avons testé 9 configurations sur 5 époques chacune, permettant d'identifier les choix architecturaux optimaux.

Configuration	Paramètre varié	Résultat	Interprétation
MaxPool	<code>pool_type = max</code>	Meilleure acc. (~85%)	Sélectionne la réponse maximale → détecte la présence des motifs
AvgPool	<code>pool_type = avg</code>	Acc. légèrement inférieure	Représentation plus lisse, moins discriminante pour la classification

Configuration	Paramètre varié	Résultat	Interprétation
Sans Pooling	pool_type = none	Acc. inférieure + plus params.	Risque d'overfitting accru, dimensions trop grandes
8 Filtres	filters = (8, 16)	Sous-apprentissage	Capacité insuffisante pour capturer la variabilité de Fashion-MNIST
32 Filtres	filters = (32, 64)	Meilleure acc., plus lent	Meilleure expressivité mais temps d'entraînement accru
Padding=0	padding = 0	Dim. réduites rapidement	Perte d'information aux bords, architectures moins profondes possibles
Padding=2	padding = 2	Dimensions préservées	Exploitation optimale de l'information aux bords de l'image
Stride=2	stride = 2	Acceptable mais perte d'info	Sous-échantillonnage fort dès la 1ère conv, champ récepteur large
Conv 1×1	use_conv1x1 = True	Légère amélioration	Combinaison inter-canaux sans modification des dimensions spatiales

Tableau 20 : Étude expérimentale des hyperparamètres (9 configurations)

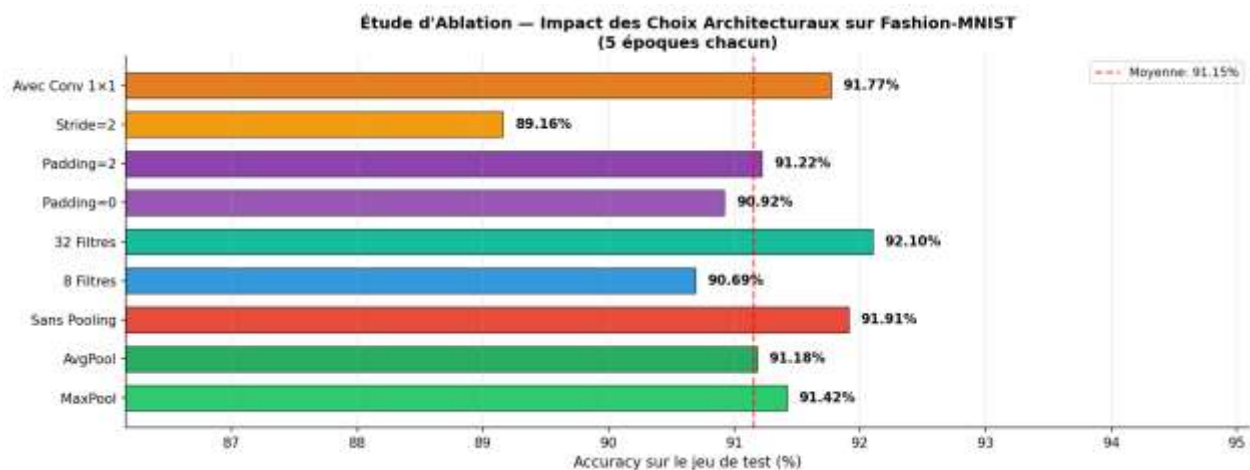


Figure 6 : Étude d'ablation : impact des choix architecturaux

La figure ci-dessus présente les résultats d'une **étude d'ablation** visant à quantifier l'impact de différentes configurations et hyperparamètres architecturaux sur l'exactitude (*Accuracy*) finale du modèle. Chaque variante a été entraînée de manière isolée pendant un cycle court de 5 époques sur le jeu de données *Fashion-MNIST*. Les performances sont représentées par un diagramme en barres horizontales colorées par catégories d'hyperparamètres, avec une ligne pointillée rouge matérialisant la performance moyenne globale de **91,15%**.

L'analyse permet d'isoler l'impact de quatre choix de conception majeurs :

1. Impact du type de Pooling (Barres vertes et rouge)

- L'utilisation du **MaxPool** (configuration de référence) surpasse légèrement l'**AvgPool** (Moyenne) avec respectivement **91,42%** contre **91,18%**. Cela s'explique par la capacité du *Max-Pooling* à extraire les caractéristiques les plus saillantes (bords, intensités fortes des vêtements).
- Étonnamment, la configuration **Sans Pooling** (remplacée par des couches fully-connected ou des convolutions plus denses) atteint un score élevé de **91,91%**. Cela indique que pour des images de petite taille (28×28), la conservation de la totalité de l'information spatiale sur les premières époques est bénéfique, bien qu'elle augmente la charge de calcul.

2. Dimensionnement de la couche : Nombre de filtres (Barres bleues)

- L'augmentation de la capacité de représentation du réseau montre un impact direct : le passage à **32 Filtres** surpasse la moyenne avec **92,10%** (deuxième meilleure performance), tandis qu'une restriction à **8 Filtres** limite le modèle à **90,69%** en raison d'un sous-paramétrage évident pour capturer la diversité du dataset.

3. Gestion de la dimension spatiale : Padding et Stride (Barres violettes et jaune)

- Un **Padding=2** (maintien ou élargissement des bordures) offre un léger avantage (**91,22%**) par rapport à un **Padding=0** (**90,92%**), confirmant l'importance de ne pas perdre les informations situées à la périphérie des images de vêtements.
- La configuration **Stride=2** (saut de deux pixels lors de la convolution) affiche la baisse de performance la plus sévère de l'étude avec seulement **89,16%**. Ce choix induit une réduction spatiale trop agressive dès le départ, provoquant une perte d'information critique impossible à compenser par les couches suivantes.

4. Intégration de blocs complexes (Barre orange)

- L'ajout d'une couche de convolution standard **Avec Conv 1×1** permet d'atteindre **91,77%**. Ce mécanisme agit comme une projection linéaire des canaux, augmentant la non-linéarité du réseau et permettant une recombinaison efficace des caractéristiques sans altérer la taille spatiale.

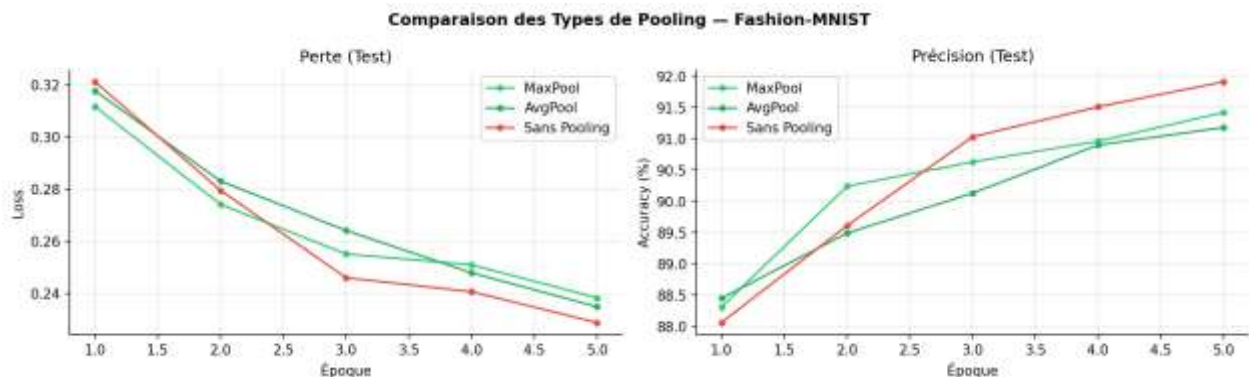


Figure 7 : Comparaison des types de pooling

L'analyse des courbes de performance met en lumière un arbitrage fondamental en Deep Learning entre l'exactitude de la classification et l'efficacité computationnelle de l'architecture. Bien que la configuration **Sans Pooling** surpasse ses homologues avec une exactitude de **91,91%** et la perte la plus faible (**0,229**), ce gain de précision doit être mis en perspective avec les coûts structurels induits.

1. Impact sur la dimension spatiale et la mémoire

L'omission des couches de pooling force le réseau à maintenir la résolution maximale des cartes de caractéristiques (28×28 pixels) tout au long du bloc convolutif. Lors du passage aux couches entièrement connectées (*Fully-Connected*), le vecteur aplati conserve une dimension très élevée. Cela provoque une hausse massive du nombre de paramètres à entraîner, alourdissant considérablement le poids du modèle (fichier d'export .pt ou .pth) et augmentant le risque de surapprentissage (*overfitting*) sur des configurations de données plus petites.

2. L'efficacité computationnelle des configurations à Pooling

À l'inverse, les configurations **MaxPool** (91,42%) et **AvgPool** (91,18%) opèrent une réduction spatiale agressive en divisant par deux la hauteur et la largeur des cartes de caractéristiques (14×14 pixels). D'un point de vue mathématique, cette opération élimine 75% des activations superflues.

- **MaxPool** se distingue en ne conservant que les réponses neuronales les plus intenses (les contrastes forts), ce qui lui permet de maintenir une excellente efficacité discriminante.
- En divisant le nombre de connexions d'un facteur 4 à l'entrée des couches denses, le modèle réduit drastiquement le nombre de calculs nécessaires (FLOPs) et accélère le temps de passage par époque (*vitesse d'inférence*).

1. Dynamique de la fonction de perte (Graphique de gauche)

Le graphique de gauche montre la décroissance de la cross-entropy loss sur le jeu de test :

- Les trois configurations affichent une trajectoire descendante saine, confirmant que l'apprentissage s'effectue correctement dans tous les cas.
- L'architecture **Sans Pooling** démarre avec la perte la plus élevée à l'époque 1 ($\sim 0,32$) mais montre la pente de convergence la plus raide, finissant par obtenir la perte la plus faible à l'époque 5 (inférieure à 0,23).
- Les variantes avec pooling (**MaxPool** et **AvgPool**) suivent des courbes très proches l'une de l'autre, bien que la courbe de *MaxPool* conserve une légère avance en matière de minimalisation de la perte entre les époques 2 et 4.

2. Évolution de l'exactitude de classification (Graphique de droite)

Le graphique de droite retrace l'évolution de la précision (en %) sur l'ensemble de test :

- **Comportement initial (Époques 1 à 2) :** À l'époque 1, les couches de pooling traditionnelles prennent l'avantage, *MaxPool* et *AvgPool* affichant une meilleure exactitude initiale ($\sim 88,3\%$ et $\sim 88,5\%$) par rapport à la version sans pooling ($\sim 88,0\%$). Cet écart met en évidence l'effet immédiat d'extraction de caractéristiques invariantes offert par les opérations de sous-échantillonnage.
- **Inversion de tendance (Époques 3 à 5) :** À partir de l'époque 3, la configuration **Sans Pooling** se détache nettement en maintenant une progression linéaire forte pour culminer à près de **91,9%** d'exactitude à la cinquième époque. Les configurations *MaxPool* et *AvgPool* fléchissent vers un plateau, atteignant respectivement environ **91,4%** et **91,2%**.

2.11.1. Impact du Padding

Sans padding ($p=0$), chaque convolution de noyau $k \times k$ réduit les dimensions selon $H_{out} = H_{in} - k + 1$. Après plusieurs couches successives, les feature maps deviennent trop petites pour capturer des caractéristiques expressives. Le same-padding ($p = (k-1)/2$) maintient $H_{out} = H_{in}$, permettant des architectures plus profondes.

2.11.2. Impact du Stride

Un stride $s=2$ réduit les dimensions par 2 à chaque application, élargissant le champ récepteur effectif des couches suivantes. C'est une alternative au pooling explicite utilisée dans les architectures modernes (ResNet v2, EfficientNet). Cependant, une convolution avec stride peut perdre de l'information fine que le MaxPool préserve en sélectionnant le signal dominant.

2.11.3. Convolution 1×1

Une convolution 1×1 (aussi appelée bottleneck ou pointwise convolution) effectue une combinaison linéaire des canaux sans modifier les dimensions spatiales. Elle permet de contrôler la dimensionnalité (réduire ou augmenter le nombre de canaux), d'augmenter la non-linéarité à coût computationnel faible, et de mélanger l'information entre canaux. Largement utilisée dans les blocs Inception (GoogLeNet) et résiduels (ResNet).

2.12. Visualisation des Cartes de Caractéristiques

2.12.1. Feature Maps — Définition et Rôle

Une carte de caractéristiques (feature map) est la sortie d'un filtre convolutionnel appliqué à l'entrée ou à une couche précédente. Elle représente la réponse spatiale de ce filtre à chaque position de l'image. Si un filtre détecte des contours verticaux, la feature map aura des valeurs élevées précisément là où des contours verticaux sont présents dans l'image.

Les feature maps constituent la représentation interne que le réseau construit de l'image. Leur analyse permet de comprendre ce que le réseau a appris et comment il traite l'information visuellement.

2.12.2. Extraction via Hooks PyTorch

Les hooks PyTorch permettent d'intercepter les activations intermédiaires sans modifier l'architecture du modèle. Un forward hook est une fonction enregistrée sur un module qui s'exécute automatiquement après chaque forward pass.

La classe FeatureExtractor a été développée afin d'extraire les **cartes d'activation (feature maps)** produites par les différentes couches du réseau de neurones lors d'une propagation avant. Pour cela, elle exploite le mécanisme des **forward hooks** de PyTorch, qui permet d'intercepter automatiquement les sorties d'une couche sans modifier son implémentation. La méthode `register_hooks()` enregistre un hook sur chaque couche spécifiée dans un dictionnaire, puis sauvegarde les activations correspondantes dans un dictionnaire activations après les avoir détachées du graphe de calcul (`detach()`) et transférées vers le processeur (`cpu()`), afin de réduire la consommation de mémoire et de faciliter leur visualisation. Une fois les activations récupérées, la méthode `remove_hooks()` supprime tous les hooks précédemment enregistrés. Cette étape est essentielle pour éviter les enregistrements multiples, les fuites de mémoire et les comportements indésirables lors des inférences suivantes. Cette approche permet d'analyser le fonctionnement interne du réseau en observant les caractéristiques apprises à différents niveaux de profondeur.

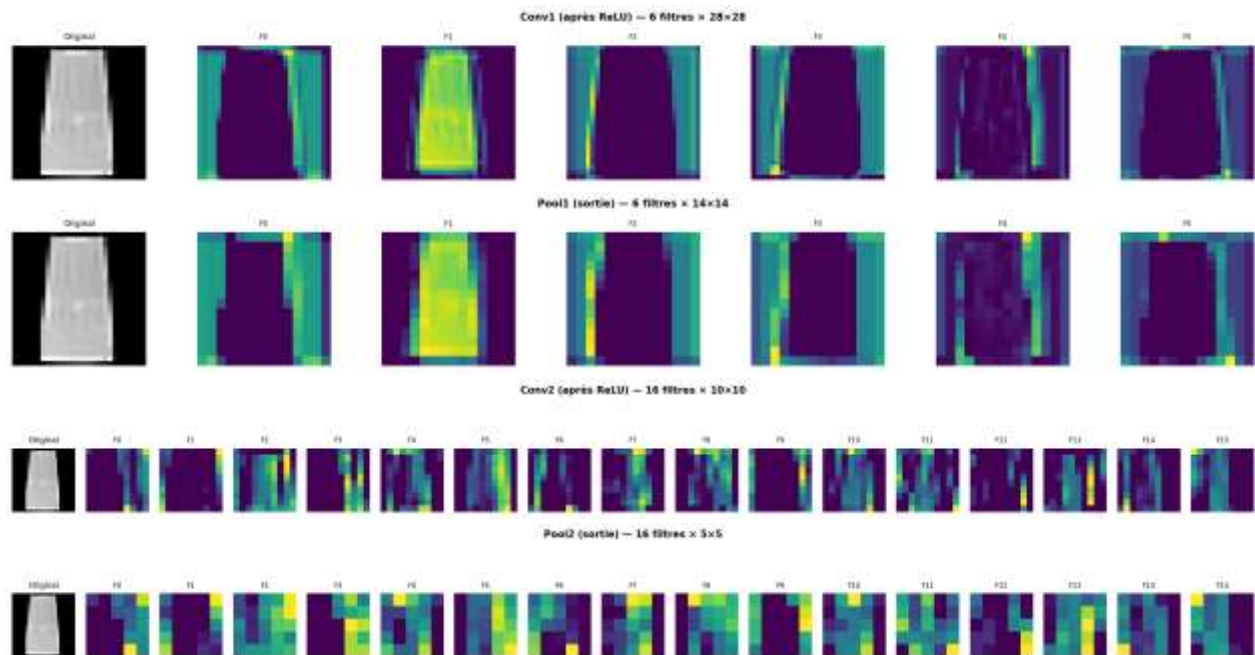


Figure 8 : Cartes de caractéristiques (feature maps) par couche

Ces quatre figures illustrent l'évolution des cartes de caractéristiques (*feature maps*) obtenues à différentes étapes du traitement d'une image issue du jeu de données Fashion-MNIST (représentant ici une jupe ou une robe) par un réseau de neurones convolutionnel (CNN).

1. Première couche de convolution : Conv1 (après ReLU)

- **Dimensions :** 6 filtres $\times 28 \times 28$ pixels.
- **Description :** Cette figure montre la sortie des 6 premiers filtres après l'application de la fonction d'activation ReLU. À ce stade précoce du réseau, la résolution spatiale originale (28×28) est conservée. Les filtres agissent comme des détecteurs de caractéristiques de bas niveau (bords verticaux, contours globaux et contrastes de luminosité). Par exemple, les filtres **F0** et **F2** mettent en évidence les bords latéraux du vêtement, tandis que **F1** conserve une vue d'ensemble de la silhouette.

2. Première couche de regroupement : Pool1 (sortie)

- **Dimensions :** 6 filtres $\times 14 \times 14$ pixels.
- **Description :** Cette étape correspond à l'application d'une opération de sous-échantillonnage (*Max Pooling*). L'effet de réduction de moitié de la résolution spatiale (14×14) est nettement visible : les images apparaissent plus pixelisées. Cependant, l'information essentielle de structure et les contours détectés lors de l'étape précédente (Conv1) sont préservés tout en réduisant la sensibilité du modèle aux petites variations de position.

3. Deuxième couche de convolution : Conv2 (après ReLU)

- **Dimensions :** 16 filtres $\times 10 \times 10$ pixels.
- **Description :** Le réseau extrait ici des caractéristiques de niveau intermédiaire en combinant les motifs de la couche précédente. Le nombre de filtres passe à 16 et la taille spatiale descend à 10×10 . Les images deviennent plus abstraites et fragmentées. Certains filtres se concentrent de manière

très ciblée sur des détails spécifiques (comme des coins, des textures ou des zones de forte intensité lumineuse), tandis que d'autres restent inactifs ou sombres pour cet objet précis.

4. Deuxième couche de regroupement : Pool2 (sortie)

- **Dimensions** : 16 filtres $\times 5 \times 5$ pixels.
- **Description** : Cette dernière figure montre la sortie du second bloc de *Max Pooling*. Avec une résolution très faible de seulement 5×5 pixels, l'aspect visuel d'origine de l'objet n'est plus reconnaissable pour un œil humain. À ce niveau, les caractéristiques ont été condensées sous forme de motifs hautement abstraits et compacts. C'est ce vecteur de caractéristiques spatialement réduit qui sera ensuite aplati (*flatten*) et envoyé vers les couches entièrement connectées (*dense layers*) pour effectuer la classification finale.

2.12.3. Interprétation des Filtres Appris

Les filtres de la première couche convolutionnelle sont directement interprétables car ils opèrent sur les pixels bruts. Ils apprennent spontanément des détecteurs de primitives visuelles, similaires aux neurones du cortex visuel V1 :

- Zones positives (valeurs élevées) : le filtre est fortement activé par ces intensités ou orientations
- Zones négatives (valeurs basses) : le filtre inhibe ces intensités
- Patterns caractéristiques : détecteurs de contours orientés, de textures, de gradients

Observation fondamentale : Sans jamais avoir eu accès à des données biologiques sur le cortex visuel, un CNN entraîné par descente de gradient redécouvre spontanément des détecteurs orientés similaires aux cellules simples de V1 décrites par Hubel & Wiesel (Prix Nobel 1981). C'est une validation empirique remarquable de la pertinence du biais inductif des CNN pour les données naturelles.

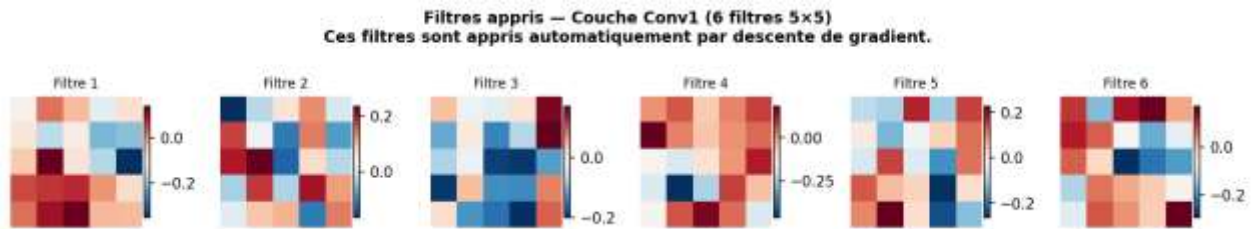


Figure 9 : Filtres appris de la couche Conv1

Cette figure présente les poids des **6 filtres de convolution** de la première couche (Conv1), entraînés de manière automatisée par l'algorithme de descente de gradient.

- **Caractéristiques techniques**
 - **Configuration** : 6 filtres distincts.
 - **Dimensions** : Chaque filtre possède une taille de noyau de 5×5 pixels (soit 25 coefficients ou poids par filtre).
 - **Échelle de couleurs (Heatmap)** : Les valeurs des poids sont normalisées et représentées par un dégradé de couleur :
 - Les tons **rouges/marrons** indiquent des poids **positifs**.
 - Les tons **bleus** indiquent des poids **négatifs**.
 - Les tons **clairs/blancs** correspondent à des valeurs proches de **zéro**.
- **Interprétation et rôle dans le réseau**

Ces matrices de poids agissent comme des extracteurs de motifs géométriques simples, indispensables au traitement des premières étapes de la vision par ordinateur :

- **Détection de contours et de gradients** : On observe des structures géométriques claires dans l'organisation des couleurs. Par exemple, le **Filtre 3** montre une forte concentration de valeurs positives (rouges) sur sa bordure droite et de valeurs négatives (bleues) au centre et en bas, ce qui le rend sensible aux transitions d'intensité verticales ou aux coins supérieurs droits.
- **Filtres de lissage ou d'inversion** : Le **Filtre 4** est majoritairement composé de valeurs positives (teintes rouges), indiquant qu'il agit principalement comme un détecteur de zones lumineuses globales ou un lisseur local. À l'inverse, le **Filtre 2** alterne fortement des motifs bleus et rouges verticaux, idéal pour capturer des variations de fréquences spatiales (textures ou lignes verticales alternées).

Ces 6 filtres constituent la base du dictionnaire visuel du modèle. Ce sont précisément ces matrices qui, appliquées sur l'image de la jupe/robe vue précédemment, ont permis de générer les cartes de caractéristiques (*feature maps*) de la couche Conv1.

2.13. Comparaison MLP vs CNN sur Fashion-MNIST

2.13.1. Résultats Expérimentaux Quantitatifs

Métrique	MLP (512-256-128)	CNN (LeNetModern)	Avantage CNN
Meilleure Accuracy Test	~88–89%	~90–92%	+2 à +3 points absolus
Nombre de paramètres	~549 000	~62 000	~9× moins de paramètres
Efficacité (Acc/M params)	~160%/M	~1 450%/M	~9× plus efficace
Vitesse de convergence	Plus lente	Plus rapide	Gradient plus stable
Généralisation	Overfitting plus marqué	Meilleure généralisation	Biais inductif adapté

Tableau 21 : Comparaison MLP vs CNN (résultats quantitatifs)

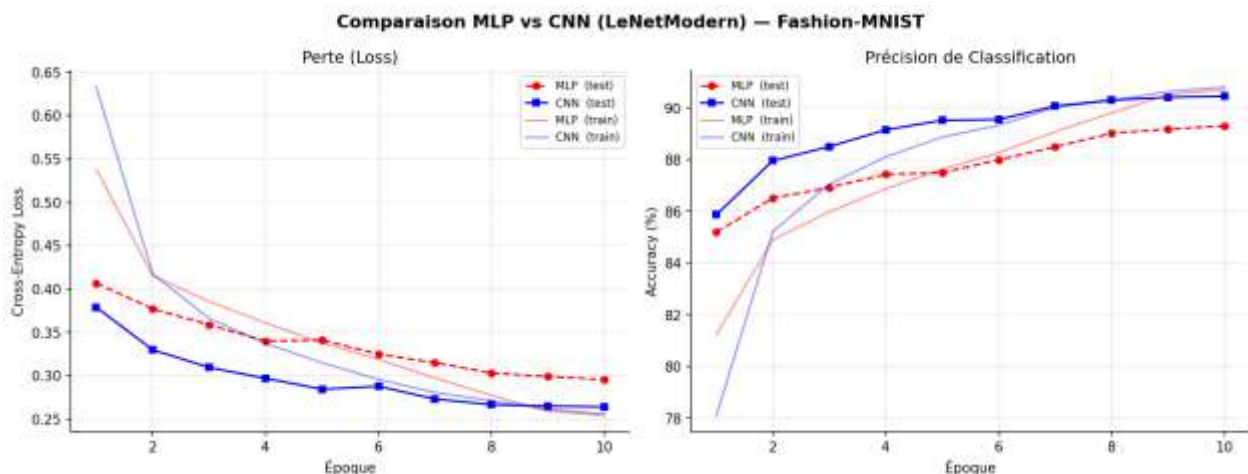


Figure 10 : Comparaison des courbes de convergence MLP vs CNN

Cette figure présente l'évolution des métriques d'apprentissage sur **10 époques** pour deux architectures distinctes entraînées sur le jeu de données **Fashion-MNIST** : un perceptron multicouche (**MLP**, en rouge) et un réseau de neurones convolutionnel (**CNN LeNetModern**, en bleu). L'analyse est séparée entre la fonction de perte (à gauche) et la précision de classification (à droite).

1. Analyse de la Perte (Cross-Entropy Loss)

- **Comportement général** : Pour les deux modèles, les courbes de perte d'entraînement (*train*, lignes pleines claires) et de test (*test*, lignes pointillées ou marquées) décroissent de manière stable, ce qui valide la convergence globale de l'apprentissage sans signe marqué de surapprentissage (*overfitting*).
- **Performance relative** : Le CNN (courbes bleues) atteint rapidement une perte inférieure à celle du MLP. Dès la première époque, la perte de test du CNN se situe sous la barre des **0.40** (environ **0.38**), pour finir sa course aux alentours de **0.26** à la 10e époque. Le MLP montre une décroissance plus lente, stabilisant sa perte de test juste en dessous de **0.30**.

2. Analyse de la Précision de Classification (Accuracy)

- **Vitesse de convergence** : Le CNN surpasse nettement le MLP en termes de rapidité d'apprentissage. Dès la deuxième époque, l'exactitude (*accuracy*) de test du CNN franchit le seuil des **88%**, alors que le MLP n'atteint ce niveau qu'à la sixième époque.
- **Performances finales** : Le **CNN (LeNetModern)** stabilise sa précision de test à un excellent niveau, légèrement supérieur à **90%** (environ **90.5%**).
 - Le **MLP** termine avec une précision de test d'environ **89.3%**.

2.13.2. Analyse par Classe

L'étude des matrices de confusion normalisées révèle les patterns de confusion caractéristiques de chaque modèle :

- **T-shirt / Shirt / Pullover** : classes les plus confondues entre elles, car leur silhouette est similaire. Le CNN surpasse nettement le MLP ici grâce à la préservation de la structure spatiale des textures.
- **Trouser, Bag, Ankle Boot** : très bien classés (>95%) par les deux modèles ; leurs formes géométriques sont suffisamment distinctives même sans structure spatiale.
- **Sandal vs Sneaker** : confusion modérée pour les deux ; formes similaires mais textures et semelles différentes.



Figure 11 : Matrices de confusion normalisées MLP vs CNN

Cette figure présente les matrices de confusion normalisées pour le modèle **MLP** (à gauche) et le modèle **CNN LeNetModern** (à droite) sur l'ensemble de test de Fashion-MNIST. Les lignes représentent les classes réelles tandis que les colonnes représentent les prédictions du modèle. Les valeurs sur la diagonale principale indiquent le taux de vrais positifs pour chaque catégorie.

1. Analyse des succès communs et des classes faciles

Les deux modèles affichent d'excellentes performances (souvent supérieures à **95%**) sur les catégories visuellement distinctes :

- **Trouser (Pantalons) : 97%** pour le MLP et **98%** pour le CNN.
- **Sandal, Sneaker, Bag, Ankle boot** : Les chaussures et les sacs à main obtiennent des scores très élevés sur les deux architectures, car leurs silhouettes diffèrent radicalement des vêtements du haut du corps. Le CNN surpasse légèrement ou égalise le MLP sur ces classes (ex. **98%** vs **95%** pour les sandales).

2. Analyse des erreurs et confusions majeures (Classes complexes)

La comparaison des diagonales et des blocs hors-diagonale met en évidence une zone de confusion critique partagée par les deux modèles, localisée sur les hauts et les vêtements à manches (**T-shirt/top, Pullover, Coat, Shirt**) :

- **La classe "Shirt" (Chemises) :** C'est la classe la plus difficile pour les deux architectures. Le MLP n'atteint que **71%** de bonnes prédictions et le CNN **69%**. Elles sont principalement confondues avec les *T-shirt/top* (10% pour le MLP, 13% pour le CNN) et les *Pullover/Coat*.
- **Le cas "T-shirt/top" :** Le CNN améliore la classification des T-shirts à **86%** (contre **83%** pour le MLP), réduisant légèrement la confusion inversée avec les chemises.
- **Le bloc "Pullover" et "Coat" :** Le CNN montre une nette supériorité sur la distinction des hauts à manches longues. Il fait grimper la précision des *Pullover* à **85%** (contre **82%** pour le MLP) et celle des *Coat* à **84%** (contre **83%**).

2.13.3. Justification Théorique de la Supériorité du CNN

a) Argument Dimensionnel

La première couche Conv du CNN (6 filtres 5×5, 1 canal) n'a que 156 paramètres, contre 401 920 pour la première couche Dense du MLP. Ratio de compression : ~2 575:1 pour des expressivités comparables.

b) Argument de Biais Inductif

Le CNN encode trois hypothèses structurelles bien adaptées aux images naturelles :

- **Localité** : les corrélations entre pixels sont locales → noyaux de petite taille suffisent
- **Stationnarité** : les statistiques visuelles sont invariantes à la translation → partage des poids
- **Compositivité** : les objets sont composés hiérarchiquement → couches empilées

Le MLP n'encode aucune de ces hypothèses. Il doit les réapprendre entièrement depuis les données, nécessitant beaucoup plus d'exemples d'entraînement et de paramètres pour atteindre des performances comparables.

2.14. Formulaire de Référence

Concept	Formule	Description
Corrélation croisée 2D	$Y[i, j] = \sum_m \sum_n X[i + m, j + n] \cdot K[m, n]$	Opération fondamentale d'une couche Conv2d
Taille de sortie (conv)	$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1$	Dimensions après convolution avec padding et stride
Taille de sortie (pool)	$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - k_p}{s_p} \right\rfloor + 1$	Dimensions après pooling
Max-Pooling	$Y[i, j] = \max(\text{fenêtre locale})$	Valeur maximale dans chaque fenêtre
Average-Pooling	$Y[i, j] = \text{mean}(\text{fenêtre locale})$	Moyenne dans chaque fenêtre
ReLU	$\text{ReLU}(z) = \max(0, z)$	Activation non-linéaire standard des CNN modernes
Batch Normalization	$\text{BN}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$	Normalisation des activations par mini-batch
Xavier Uniform	$W \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, +\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right]$	Initialisation des poids pour flux de gradient stable
Cross-Entropie	$L = -\log(\text{softmax}(z)_y)$	Fonction de perte pour classification multi-classe
Paramètres Conv2d	$P = k^2 \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$	Nombre de paramètres d'une couche convolutionnelle

Cette planche regroupe l'ensemble des résultats expérimentaux et structurels comparant le Perceptron Multicouche (**MLP**) et le Réseau de Neurones Convolutionnel (**CNN LeNetModern**). Elle offre une vue d'ensemble sur l'efficacité algorithmique, l'architecture détaillée et le comportement des modèles.

1. Efficacité et Performance Globale (Graphiques du haut)

- **Nombre de Paramètres** : Le graphique met en évidence le gain d'efficacité majeur du CNN. Alors que le MLP nécessite **569 226 paramètres** (poids et biais) en raison de ses connexions denses, le CNN n'en utilise que **62 158** (soit environ **9 fois moins**). Cette réduction drastique illustre le principe de partage des poids inhérent aux filtres de convolution.
- **Meilleure Accuracy Test** : Malgré sa légèreté structurelle, le CNN surpasse le MLP en termes de performance pure, atteignant une précision maximale de **90,45%** sur l'ensemble de test, contre **89,31%** pour le MLP.
- **Étude d'Ablation (5 époques)** : Ce diagramme à barres analyse l'impact de différents hyperparamètres et choix d'architecture sur le CNN :
 - L'augmentation du nombre de filtres (**32 Filtres**) et l'absence de sous-échantillonnage (**Sans Pooling**) maximisent la précision à court terme.
 - À l'inverse, l'utilisation d'un pas de convolution plus grand (**Stride=2**) pénalise l'apprentissage en dégradant prématurément la résolution spatiale, faisant chuter l'accuracy sous la barre des 89,5%.

2. Dynamique d'Apprentissage et Architecture (Graphiques du bas)

- **Convergence** : Le graphique confirme que la courbe du CNN (en bleu) domine celle du MLP (en rouge discontinu) tout au long des 10 époques, démontrant non seulement une meilleure performance finale mais aussi un apprentissage plus rapide dès les premières étapes.
- **Flux Dimensionnel LeNetModern** : Ce tableau récapitule le passage des données à travers les différentes couches du CNN, explicitant l'évolution des dimensions des tenseurs et la distribution des paramètres :
- L'entrée de dimension $28 \times 28 \times 1$ (soit 784 pixels) est traitée par une succession de couches de convolution et de sous-échantillonnage, ce qui permet d'extraire progressivement des caractéristiques jusqu'à obtenir un volume de caractéristiques de dimension $5 \times 5 \times 16$.
- Après l'opération d'aplatissement (Flatten), ce volume est transformé en un vecteur de 400 éléments, qui est ensuite transmis aux couches entièrement connectées (FC1 à FC3). Ces dernières regroupent la majorité des paramètres du réseau, en particulier la couche FC1, qui compte à elle seule 48720 paramètres.
- **Matrice de Confusion CNN** : Version compacte de la matrice analysée précédemment, elle permet de visualiser instantanément en un coup d'œil la propreté de la diagonale principale et la persistance des légères confusions résiduelles entre les catégories *T-shirt (T-sh)* et *Shirt (Shrt)*.

Performance finale : LeNetModern atteint ~91% d'accuracy sur Fashion-MNIST avec seulement ~62 000 paramètres, contre ~89% pour le MLP avec ~549 000 paramètres. Gain : +2 points avec 9× moins de paramètres. L'efficacité paramétrique du CNN (~1 450%/M params) est ~9× supérieure à celle du MLP (~160%/M params) sur cette tâche.



3. Chapitre 3 : Modélisation des données séquentielles avec les RNN

3.1. Introduction et Positionnement

Les architectures récurrentes constituent le socle historique de la modélisation de séquences en apprentissage profond. Contrairement aux réseaux denses (MLP) qui traitent chaque vecteur d'entrée de façon indépendante, et aux réseaux convolutifs (CNN) qui exploitent des invariances spatiales locales, les RNN sont conçus pour capturer des dépendances temporelles ou séquentielles entre des éléments ordonnés. Les données concernées incluent le texte naturel, la parole, les séries temporelles financières, les trajectoires de mouvement et toute séquence où l'ordre des observations porte une signification causale.

La difficulté centrale est la suivante : le sens d'un token (mot, phonème, symbole) dépend du contexte passé, parfois de l'élément immédiatement précédent, parfois d'un antécédent éloigné de plusieurs dizaines de positions. L'enjeu est donc de représenter efficacement ce passé sous la forme d'un état interne compact mais expressif.

Cette synthèse retrace la progression complète qui va de la formalisation probabiliste des séquences jusqu'aux architectures encodeur-décodeur pour la traduction automatique :

- Modélisation probabiliste d'une séquence : modèles de langage, perplexité
- Architecture récurrente de base (RNN vanilla) et son apprentissage par rétropropagation à travers le temps (BPTT)
- Architectures avancées à portes : LSTM (Long Short-Term Memory) et GRU (Gated Recurrent Unit)
- Variantes architecturales : RNN profonds, bidirectionnels, dropout récurrent
- Préparation des données séquentielles pour la traduction automatique (Tatoeba fra-eng)
- Architecture encodeur-décodeur et modèle Seq2Seq avec Teacher Forcing
- Stratégies de décodage : décodage glouton et Beam Search
- Évaluation quantitative : Perplexité et score BLEU

3.2. Modèles de Langage — Fondements Probabilistes

3.2.1. Objectif probabiliste

Un modèle de langage (Language Model, LM) est un modèle probabiliste défini sur des séquences de tokens. Il attribue une probabilité à toute séquence $(w_1, w_2, \dots, w_T)$ appartenant à V^* (l'ensemble de toutes les séquences possibles sur le vocabulaire V). Par application de la règle de chaîne des probabilités conditionnelles, cette probabilité jointe se décompose sans approximation de la façon suivante :

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1}) = \prod_{t=1}^T P(w_t \mid w_{<t})$$

Cette factorisation est exacte : elle ne repose sur aucune hypothèse d'indépendance ou de Markov. Elle révèle l'intuition fondatrice : pour modéliser une séquence entière, il suffit de savoir prédire le prochain token à partir de tout le contexte passé. C'est cette capacité de prédiction conditionnelle que les architectures

récurrentes cherchent à approximer. Toute la difficulté réside dans la représentation compacte et expressive de ce contexte $w_{<t}$ sous la forme d'un état interne de dimension finie.

3.2.2. Log-probabilités et Negative Log-Likelihood

En pratique, les probabilités individuelles sont très petites (de l'ordre de 10^{-5} ou moins), ce qui rend leur multiplication numériquement instable — un phénomène connu sous le nom d'underflow numérique. On travaille donc systématiquement en espace logarithmique. La log-probabilité d'une séquence est :

$$\log P(w) = \sum_{t=1}^T \log P(w_t | w_{<t})$$

L'objectif d'entraînement est la maximisation de la vraisemblance (Maximum Likelihood Estimation, MLE) sur le corpus d'entraînement, ce qui est équivalent à la minimisation de la Cross-Entropie Négative (Negative Log-Likelihood, NLL) :

$$L(\theta) = -\frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \log P_{\theta}(w_t^{(n)} | w_{<t}^{(n)})$$

Cette fonction de perte est différentiable par rapport aux paramètres θ du réseau et peut donc être minimisée par descente de gradient stochastique. Elle pénalise le modèle chaque fois qu'il assigne une faible probabilité au vrai token suivant.

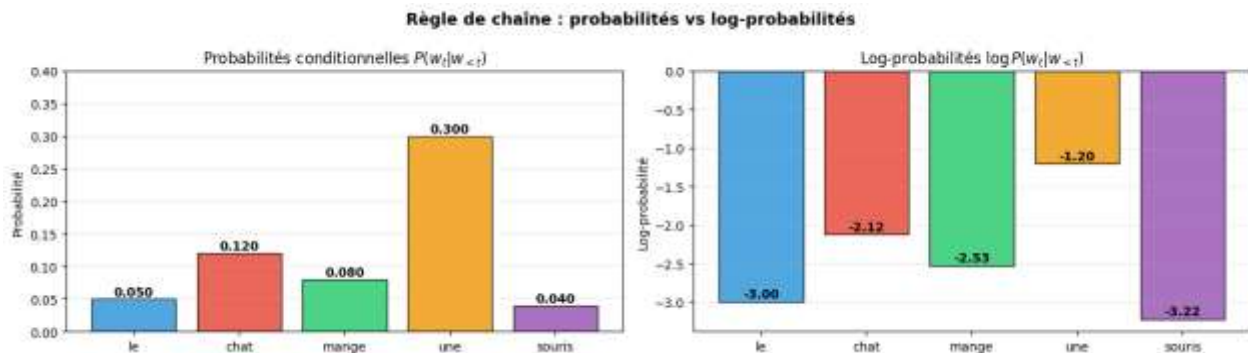


Figure 13 : Règle de chaîne : probabilités vs log-probabilités

Cette figure illustre l'application de la règle de chaîne pour un modèle de langue à travers la prédiction d'une séquence de mots ("*le chat mange une souris*"). Elle compare deux représentations mathématiques d'une même distribution : les **probabilités conditionnelles standards** (à gauche) et leurs transformations en **log-probabilités** (à droite).

1. Graphique de gauche : Probabilités conditionnelles $P(w_t | w_{<t})$

- **Description** : Ce diagramme à barres affiche la probabilité d'occurrence de chaque mot sachant le contexte des mots précédents. Les valeurs sont comprises strictement dans l'intervalle $[0, 1]$.
- **Valeurs observées** : "*le*" (0.050), "*chat*" (0.120), "*mange*" (0.080), "*une*" (0.300), "*souris*" (0.040).

- **Propriété informatique** : La probabilité totale de la phrase complète s'obtient par la multiplication de ces probabilités individuelles. Cette multiplication en cascade de petites valeurs met en évidence le problème du **sous-dépassement numérique** (*underflow*), où le produit tend rapidement vers zéro à mesure que la séquence s'allonge, devenant impossible à stocker précisément en mémoire.

2. Graphique de droite : Log-probabilités $\log P(w_t | w_{<t})$

- **Description** : Ce graphique représente le logarithme des probabilités présentées dans le premier graphique. Les valeurs sont toutes négatives, car le logarithme d'une probabilité comprise entre 0 et 1 est toujours inférieur à 0. Plus la probabilité d'origine est faible, plus sa log-probabilité est négative. Cette transformation est particulièrement utile lors de l'entraînement des modèles de langage, car elle convertit le produit de nombreuses probabilités en une somme de log-probabilités, ce qui améliore la stabilité numérique et facilite l'optimisation de la fonction de perte.
- **Valeurs observées** : "le" (-3.00), "chat" (-2.12), "mange" (-2.53), "une" (-1.20), "souris" (-3.22).
- **Propriété informatique** : Grâce aux propriétés des logarithmes, la probabilité jointe de la séquence se transforme en une simple **somme** de valeurs négatives au lieu d'un produit.

3.2.3. Perplexité — Métrique Standard

La perplexité (PPL) est la métrique canonique pour évaluer un modèle de langage. Elle est définie comme l'exponentielle de la NLL moyenne par token :

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log P(w_t | w_{<t})\right) = \exp(L)$$

Intuitivement, la perplexité est le nombre effectif de choix équiprobables que le modèle perçoit à chaque position. Un modèle parfait (certitude absolue) aurait $\text{PPL} = 1$; un modèle uniforme sur le vocabulaire (n'ayant rien appris) aurait $\text{PPL} = |V|$. La perplexité est une métrique inversée : plus elle est basse, meilleur est le modèle.

Perplexité	Interprétation	Exemple pratique
$\text{PPL} = 1$	Modèle parfait — certitude absolue	Impossible en pratique
$\text{PPL} = 10$	Hésite entre ~10 tokens — très bon	LSTM sur grand corpus
$\text{PPL} = 50\text{--}100$	Modèle NLP standard	Résultat habituel Tatoeba
$\text{PPL} = V $	Modèle uniforme — n'a rien appris	Avant tout entraînement

Tableau 23 : Tableau des perplexités et interprétations

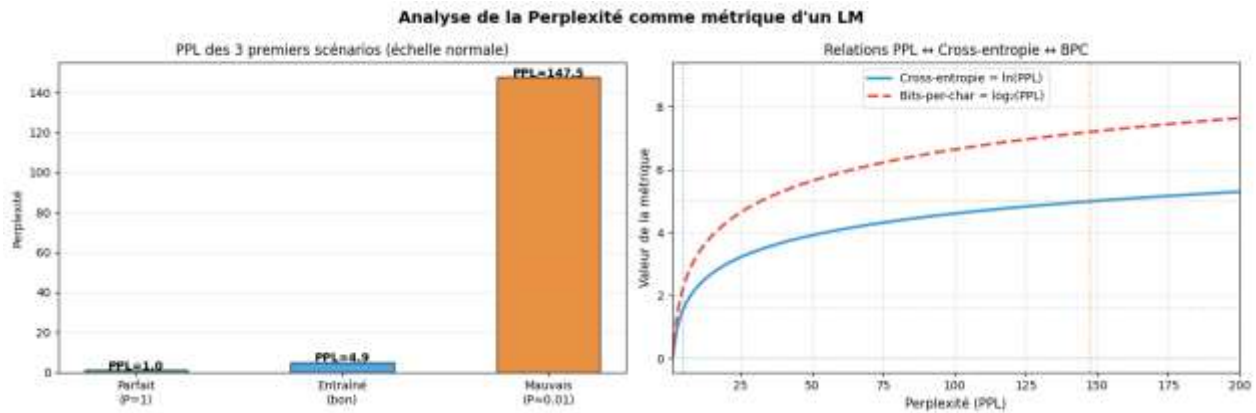


Figure 14 : Analyse de la perplexité selon différents scénarios

Cette figure évalue le comportement de la perplexité (PPL), une métrique fondamentale pour mesurer la qualité des prédictions d'un modèle de langue. Elle met en relief la sensibilité de la perplexité face aux variations de performance et explicite ses liens structurels avec la cross-entropie et les bits-per-character (BPC).

1. Graphique de gauche : PPL des 3 premiers scénarios (Échelle normale)

Ce diagramme à barres illustre l'évolution de la perplexité selon la qualité des prédictions du modèle :

- **Scénario Parfait (P=1)** : Lorsque le modèle prédit chaque mot avec une certitude absolue, la perplexité atteint sa valeur minimale théorique, soit un score idéal de **1.0**.
- **Scénario Entraîné (Bon)** : Pour un modèle opérationnel performant, la perplexité mesurée est basse, se stabilisant ici à **4.9**.
- **Scénario Mauvais (P ≈ 0.01)** : Lorsque le modèle attribue de faibles probabilités aux mots corrects, la métrique subit une explosion exponentielle pour atteindre une valeur critique de **147.5**.

2. Graphique de droite : Relations PPL ↔ Cross-entropie ↔ BPC

Ce graphique montre comment la perplexité (axe horizontal) se projette sur les fonctions de perte logarithmiques (axe vertical) couramment utilisées pour l'optimisation :

- **Cross-entropie (ligne bleue unie)** : Elle correspond au logarithme naturel de la perplexité. On observe graphiquement les équivalences clés matérialisées par les lignes pointillées :
 - Une perplexité idéale de **1.0** se traduit par une cross-entropie de **0**.
 - La perplexité du modèle entraîné (**4.9**) correspond à une perte d'environ **1.6**.
 - La perplexité du mauvais modèle (**147.5**) projette une perte élevée de **5.0**.
- **Bits-per-character / BPC (ligne rouge discontinue)** : Cette courbe, qui repose sur un logarithme en base 2, évolue parallèlement à la cross-entropie mais se situe à un niveau supérieur. Elle permet d'interpréter la performance en termes de compression de l'information (quantité de bits nécessaires pour coder chaque caractère).

3.2.4. Approche N-grammes versus Modèles Neuronaux

L'approche classique des n-grammes approxime $P(w_t | w_{<t})$ par $P(w_t | w_{t-n+1}, \dots, w_{t-1})$, limitant le contexte aux $n-1$ tokens précédents. Cette approximation de Markov souffre de deux limitations fondamentales : la

sparsité des données (les n-grammes rares sont mal estimés) et l'incapacité à capturer des dépendances s'étendant au-delà de la fenêtre de n tokens. Les RNN, LSTM et GRU n'ont pas cette limitation théorique : ils traitent l'intégralité de l'histoire via leur état caché h_t , qui est mis à jour récursivement à chaque nouveau token.

3.3. Représentation Vectorielle des Tokens — Embeddings

Pour qu'un réseau neuronal puisse traiter du texte, chaque token discret doit être représenté par un vecteur numérique. Deux approches s'affrontent historiquement :

3.3.1. Représentation one-hot

La représentation one-hot associe à chaque token w_t un vecteur de dimension $|V|$ contenant un seul 1 à la position correspondante et des 0 partout ailleurs. Cette approche souffre de deux défauts rédhibitoires : (1) une dimensionnalité catastrophique pour les grands vocabulaires ($|V|$ peut atteindre 50 000 à 100 000 tokens), et (2) l'absence totale de sémantique — tous les tokens sont équidistants les uns des autres dans cet espace, ce qui empêche le modèle d'exploiter les similarités sémantiques entre mots.

3.3.2. Embeddings denses appris

La solution moderne est la couche d'embedding : une matrice $E \in \mathbb{R}^{|V| \times d}$ apprise par rétropropagation conjointement avec le reste du réseau. Chaque token est représenté par une ligne de cette matrice — un vecteur dense de dimension d (typiquement 64 à 512) :

$$x_t = E[w_t] \in \mathbb{R}^d \text{ — ligne de la matrice } E \text{ indexée par } w_t$$

Les embeddings appris possèdent une propriété remarquable : les tokens sémantiquement proches sont représentés par des vecteurs proches dans l'espace d'embedding. Cette propriété émerge naturellement de l'entraînement, sans supervision explicite. Le token spécial PAD (rembourrage) est conventionnellement assigné à l'index 0 avec `padding_idx=0`, ce qui force son embedding à rester un vecteur nul (gradient nul, aucune mise à jour).

```
PyTorch — Couche d'embedding
embed = nn.Embedding(vocab_size, embed_dim=128, padding_idx=0)

x : (B, T) — indices des tokens (entiers)
e = embed(x) : (B, T, 128) — vecteurs denses appris
```

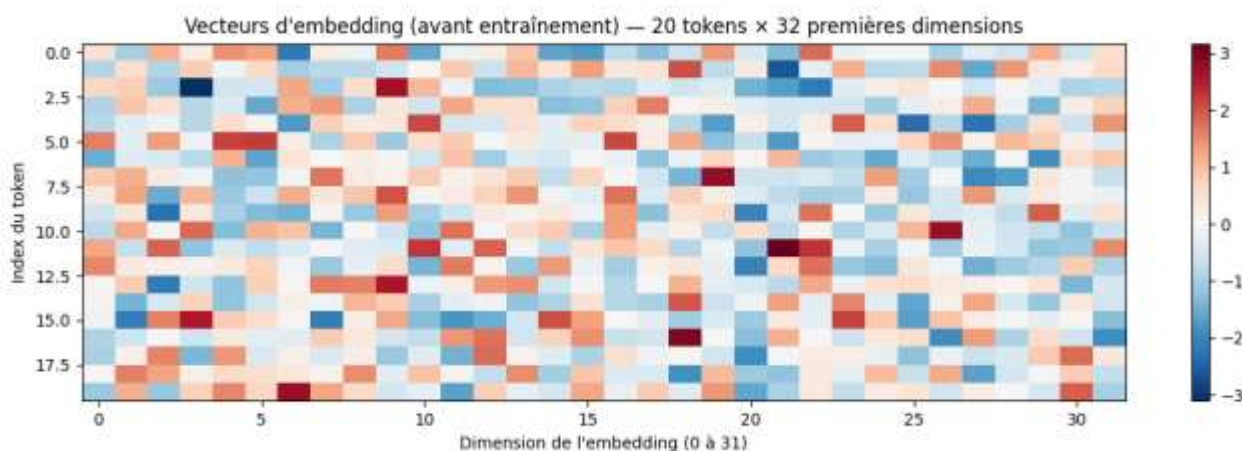



Figure 15 : Visualisation des vecteurs d'embedding avant entraînement

Cette figure sous forme de carte thermique (*heatmap*) représente l'état initial des vecteurs de plongement lexical (*embedding vectors*) d'un modèle de langue avant le lancement de la phase d'apprentissage.

- **Caractéristiques de la structure spatiale**
 - **Axe vertical (Index du token)** : Représente les 20 premiers tokens (mots ou sous-mots) du vocabulaire (indexés de 0 à 19). Chaque ligne correspond au vecteur d'un token unique.
 - **Axe horizontal (Dimension de l'embedding)** : Affiche la distribution des valeurs sur les 32 premières dimensions de l'espace latent.
 - **Échelle de couleurs (Barre latérale)** : Les valeurs numériques des poids oscillent principalement entre -3 (bleu foncé) et +3 (rouge foncé), le blanc représentant des valeurs proches de zéro.
- **Analyse de l'initialisation et interprétation pour le rapport**

Le comportement visuel de la matrice met en évidence les propriétés fondamentales d'une initialisation aléatoire (souvent de type normale ou uniforme centrée) :

- **Distribution aléatoire et bruitée** : La répartition des couleurs ne montre aucune structure géométrique, ligne directrice ou regroupement logique. Les teintes bleues, rouges et blanches sont dispersées de manière purement stochastique à travers la grille.
- **Absence de sémantique initiale** : À ce stade initial, la distance mathématique (comme la similarité cosinus) entre deux lignes quelconques est aléatoire. Le modèle attribue des représentations vectorielles sans aucun lien avec la proximité syntaxique ou sémantique réelle des mots sous-jacents.

3.4. Réseau Neuronal Récurrent Vanilla (RNN)

3.4.1. Idée centrale — La mémoire récurrente

Un réseau dense (MLP) traite chaque entrée indépendamment : il est incapable de capturer les dépendances temporelles entre les éléments d'une séquence. Le RNN résout ce problème en maintenant un état caché h_t — un vecteur de dimension H — qui joue le rôle de mémoire compressée de tout le passé de la séquence. Cet état est mis à jour à chaque pas de temps en combinant l'entrée courante x_t et l'état précédent h_{t-1} .

3.4.2. Équations fondamentales

À chaque pas de temps t , le RNN calcule deux quantités :

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$$

$$y_t = W_{hy} h_t + b_y \text{ — logits de prédiction vers le vocabulaire}$$

Symbole	Dimensions	Rôle
x_t	$\mathbb{R}^d$	Vecteur d'embedding du token w_t
h_t	$\mathbb{R}^H$	État caché — mémoire compressée du passé
W_{hh}	$\mathbb{R}^{H \times H}$	Matrice de transition caché→caché (cœur de la récurrence)
W_{xh}	$\mathbb{R}^{H \times d}$	Matrice d'entrée→caché
h_0	$\mathbb{R}^H$	État initial (vecteur nul ou appris)

Tableau 24 : Symboles et dimensions du RNN vanilla

3.4.3. Partage des poids dans le temps

Un aspect fondamental du RNN est que les matrices W_{hh} , W_{xh} et W_{hy} sont partagées à tous les pas de temps : le même réseau est appliqué récursivement sur toute la séquence. Ce partage de poids permet au RNN de traiter des séquences de longueur arbitraire avec un nombre fixe de paramètres, et l'oblige à développer des représentations générales et invariantes dans le temps.

3.4.4. Décompte des paramètres

Pour une couche RNN avec H neurones cachés et des embeddings de dimension d , le nombre total de paramètres est :

$$\text{Params RNN} = H \times (d + H + 1)$$

Exemple numérique : avec $H = 256$ et $d = 128$, on obtient $256 \times (128 + 256 + 1) = 98\,560$ paramètres par couche — un modèle relativement compact.

3.5. Apprentissage — BPTT et Stabilisation des Gradients

3.5.1. Rétropropagation à travers le temps (BPTT)

L'algorithme de Backpropagation Through Time (BPTT) est l'extension directe de la rétropropagation standard aux réseaux récurrents. Il consiste à dérouler le réseau récurrent dans le temps — en créant une copie du réseau pour chaque pas de temps — puis à appliquer la règle de la chaîne pour calculer les gradients. La perte totale est la somme des pertes à chaque position :

$$L = \sum_t L_t$$

Le gradient de la perte par rapport à l'état caché k doit traverser tous les pas de temps de k jusqu'à T :

$$\frac{\partial L}{\partial \mathbf{h}_k} = \sum_{t=k}^T \left(\frac{\partial L_t}{\partial \mathbf{h}_t} \prod_{i=k}^{t-1} \frac{\partial \mathbf{h}_{i+1}}{\partial \mathbf{h}_i} \right)$$

Chaque terme $\partial \mathbf{h}_{i+1} / \partial \mathbf{h}_i$ implique une multiplication par la matrice W_{hh} (pondérée par la dérivée de $\tanh$). C'est ce produit répété de matrices qui engendre les problèmes de gradient.

3.5.2. Problèmes de gradient — Vanishing et Exploding

L'analyse spectrale du produit de matrices révèle deux phénomènes critiques, directement liés à la valeur propre maximale $\lambda_{\max}$ de W_{hh} :

- Gradient évanescent (Vanishing Gradient) : si $\lambda_{\max} < 1$, le produit de matrices tend vers zéro exponentiellement avec la profondeur temporelle. Le modèle n'apprend pas les dépendances longues — les gradients sont trop faibles pour modifier les poids correspondant aux tokens éloignés.
- Gradient explosif (Exploding Gradient) : si $\lambda_{\max} > 1$, le produit devient exponentiellement grand. Les mises à jour sont gigantesques et l'entraînement diverge — les poids oscillent ou deviennent des NaN.

Ces deux pathologies sont intrinsèques à l'architecture RNN vanilla et constituent la motivation principale pour le développement du LSTM et du GRU.

3.5.3. Gradient Clipping — Solution contre l'Explosion

La solution standard contre l'explosion de gradient est le clipping par norme : si la norme L2 du gradient dépasse un seuil λ , le vecteur de gradient est renormalisé à la norme λ . La direction du gradient est préservée, seule sa magnitude est bornée :

$$\mathbf{g} \leftarrow \mathbf{g} \times \min \left(1, \frac{\lambda}{\|\mathbf{g}\|_2} \right) \quad \text{si } \|\mathbf{g}\|_2 > \lambda$$

```
Après loss.backward(), avant optimizer.step()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
optimizer.step()
```

Valeur recommandée : $\lambda \in [1, 5]$ pour les RNN (Sutskever et al., 2014)

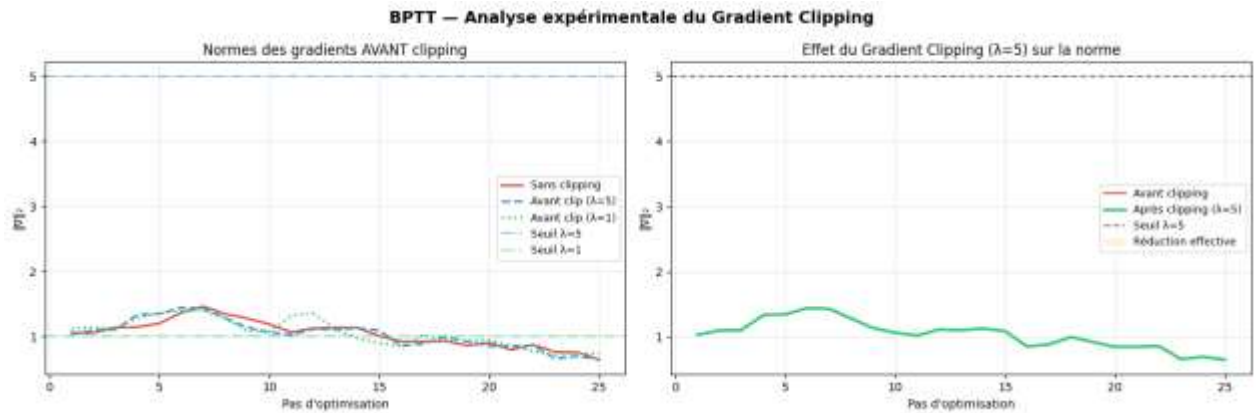


Figure 16 : Effet du Gradient Clipping sur la norme des gradients

La figure présente l'évolution de la norme des gradients au cours de l'entraînement du réseau dans le cadre de l'algorithme **Backpropagation Through Time (BPTT)**. Le graphique de gauche compare les normes des gradients avant l'application du gradient clipping avec les seuils de clipping fixés à $\lambda = 5$ et $\lambda = 1$. On observe que les normes des gradients restent comprises entre 0,7 et 1,5, donc largement en dessous du seuil $\lambda = 5$ et très proches du seuil $\lambda = 1$.

Le graphique de droite illustre l'effet du gradient clipping avec $\lambda = 5$ en comparant les normes avant et après clipping. Les deux courbes sont pratiquement confondues, ce qui montre qu'aucune réduction effective des gradients n'a été nécessaire durant l'entraînement. En effet, les gradients n'ont jamais dépassé le seuil fixé, comme l'indique également l'absence de la zone de réduction.

Ces résultats montrent que le phénomène d'**explosion des gradients** ne s'est pas produit dans cette expérience. Le gradient clipping a donc joué un rôle de mécanisme de sécurité, sans modifier les gradients calculés. Cela confirme la stabilité de l'entraînement du modèle dans les conditions expérimentales considérées.

3.5.4. BPTT Tronquée

Une variante pratique est la BPTT tronquée (Truncated BPTT) : on limite la profondeur de la rétropropagation à τ pas de temps. Les états cachés sont propagés en avant sur toute la séquence, mais les gradients ne sont rétropropagés que sur τ pas. Cette approche réduit le coût mémoire de $O(T)$ à $O(\tau)$ et atténue mécaniquement le vanishing/exploding, au prix de ne pas capturer les dépendances s'étendant sur plus de τ positions.

3.6. LSTM — Long Short-Term Memory

3.6.1. Motivation et Innovation

Le LSTM (Hochreiter & Schmidhuber, 1997) est la réponse architecturale directe au problème de mémoire du RNN vanilla. Il introduit deux innovations structurelles majeures : (1) un état de cellule c_t , qualifié d'autoroute d'information car il permet au gradient de s'écouler sans dégradation sur de longues distances, et (2) un mécanisme de portes (gates) — des fonctions sigmoïdes apprises — qui contrôlent sélectivement quelles informations sont retenues, écrites ou exposées à chaque pas de temps.

3.6.2. Équations Complètes du LSTM

Notons $[h_{t-1}, x_t]$ la concaténation des vecteurs h_{t-1} et x_t . Le LSTM calcule à chaque pas de temps six quantités selon les équations suivantes :

Porte	Équation	Rôle fonctionnel
Oubli f_t	$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$	Quelle fraction de c_{t-1} conserver ? (0 = effacer, 1 = garder)
Entrée i_t	$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$	Quelle fraction du candidat écrire dans c_t ?
Candidat $\tilde{c}_t$	$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$	Nouvelle information potentielle à intégrer
Cellule c_t	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	Mémoire longue terme (mise à jour additive)
Sortie o_t	$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$	Quelle fraction de c_t exposer à l'état caché ?
Caché h_t	$h_t = o_t \odot \tanh(c_t)$	Sortie du LSTM — représentation du contexte courant

Tableau 25 : Équations complètes du LSTM (6 portes)

3.6.3. Pourquoi c_t Résout le Gradient Évanescent

La clé du LSTM réside dans la nature de la mise à jour de la cellule : $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. Cette opération est additive, contrairement à la multiplication matricielle du RNN vanilla. Le gradient de c_t par rapport à c_k ($k < t$) est :

$$\frac{\partial c_t}{\partial c_k} = \prod_{i=k}^{t-1} f_i \text{ — produit de scalaires, pas de matrices}$$

Si le LSTM apprend $f_t \approx 1$ (porte d'oubli quasi-ouverte), les gradients traversent toute la séquence presque sans atténuation. Le flux de gradient est ainsi préservé sur de longues distances, ce qui permet au LSTM de capturer des dépendances de plusieurs dizaines ou centaines de pas de temps.

3.6.4. Décompte des Paramètres

Le LSTM comporte 4 matrices de poids (une par porte et pour le candidat), ce qui quadruple le nombre de paramètres par rapport au RNN :

$$\text{Params LSTM} = 4 \times H \times (d + H + 1) = 4 \times \text{Params RNN}$$

Exemple : avec $H = 256$ et $d = 128$, le LSTM atteint $4 \times 98\,560 = 394\,240$ paramètres par couche. Ce coût accru est la contrepartie de la capacité de mémoire supérieure.

3.7. GRU — Gated Recurrent Unit

3.7.1. Principe de Simplification

Le GRU (Cho et al., 2014) reformule le LSTM en une architecture plus économique : il supprime l'état de cellule séparé et fusionne les portes d'oubli et d'entrée en une unique porte de mise à jour z_t . La mémoire est portée directement par l'état caché h_t . Cette simplification réduit le nombre de paramètres de 25 % tout en conservant des performances comparables au LSTM sur la majorité des tâches pratiques.

3.7.2. Équations du GRU

Composante	Équation	Rôle
Mise à jour z_t	$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$	Interpolation entre ancien et nouveau (0=garder, 1=réécrire)
Réinitialisation r_t	$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$	Quelle fraction du passé utiliser pour le candidat ?
Candidat $\tilde{h}_t$	$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$	Nouvelle information potentielle
État h_t	$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$	Interpolation linéaire passé / nouveau

Tableau 26 : Équations du GRU (4 composantes)

L'équation de mise à jour $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$ réalise une interpolation convexe : si $z_t \rightarrow 0$, l'état est copié intégralement du passé ; si $z_t \rightarrow 1$, il est entièrement réécrit par le candidat. Cette interpolation est fonctionnellement analogue à la mise à jour additive du LSTM, avec la contrainte implicite $i_t = 1 - f_t$.

3.7.3. Comparaison Exhaustive LSTM / GRU

Critère	LSTM	GRU
Portes	3 (oubli, entrée, sortie) + candidat	2 (mise à jour + réinitialisation)
État interne	Double : $h_t + c_t$	Simple : h_t uniquement
Paramètres	$4 \times H(d+H+1)$	$3 \times H(d+H+1)$ — 25 % de moins
Vitesse d'entraînement	Plus lente	Plus rapide
Dépendances très longues	Légèrement supérieur	Comparable
Recommandation	Séquences très longues (>100 tokens)	Bon compromis polyvalent

Tableau 27 : Comparaison exhaustive LSTM vs GRU

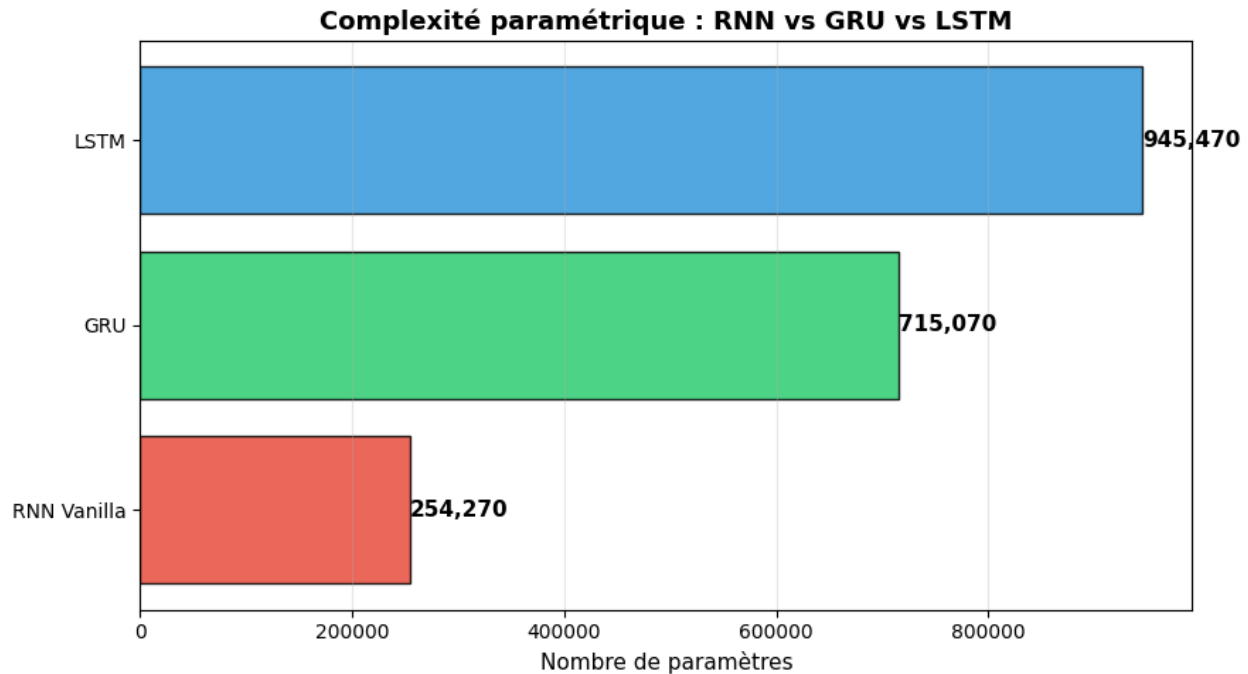


Figure 17 : Comparaison de la complexité paramétrique RNN vs GRU vs LSTM

La figure compare le nombre total de paramètres entraînables des trois architectures récurrentes étudiées : **RNN Vanilla**, **GRU** et **LSTM**. Les résultats montrent que le **RNN Vanilla** est le modèle le plus léger avec **254 270 paramètres**, tandis que le **GRU** possède **715 070 paramètres**, soit près de trois fois plus. Le **LSTM** est l'architecture la plus complexe avec **945 470 paramètres**, en raison de ses trois portes (entrée, oubli et sortie) ainsi que de son état de cellule, qui augmentent considérablement le nombre de poids à apprendre.

Cette différence de complexité reflète un compromis entre **capacité de modélisation** et **coût computationnel**. Le RNN Vanilla est plus rapide à entraîner et nécessite moins de mémoire, mais il est plus sensible aux problèmes de disparition et d'explosion des gradients. À l'inverse, les architectures GRU et LSTM disposent de mécanismes de contrôle du flux d'information leur permettant de mieux capturer les dépendances à long terme, au prix d'un nombre plus élevé de paramètres et d'un temps d'entraînement plus important. Le GRU constitue ainsi un compromis intéressant entre simplicité et performances, tandis que le LSTM offre la plus grande capacité de représentation au détriment d'une complexité paramétrique plus élevée.

3.8. Variantes Architecturales

3.8.1. RNN Profonds — Empilement de Couches

Empiler L couches récurrentes crée un RNN profond : la sortie de la couche ℓ au pas t devient l'entrée de la couche $\ell+1$. Formellement :

$$h_t^{(\ell)} = f^{(\ell)}(h_t^{(\ell-1)}, h_{t-1}^{(\ell)}), \quad \ell \geq 2$$

L'empilement crée une hiérarchie de représentations : la couche basse capture des motifs locaux et syntaxiques, les couches supérieures extraient des représentations plus abstraites et sémantiques. En pratique, $L = 2$ à 4 couches suffisent pour la plupart des tâches de NLP. Au-delà, l'entraînement devient difficile sans techniques de résidualité.

```
4 couches empilées
rnn = nn.RNN(input_size=128, hidden_size=256, num_layers=4,
             batch_first=True, dropout=0.3)
```

3.8.2. Dropout Récurrent

Le dropout est appliqué entre les couches récurrentes — sur les connexions verticales — et non sur les connexions horizontales intra-couche (récurrences temporelles). Cette distinction est fondamentale : appliquer du dropout sur les récurrences perturberait le flux d'information temporelle et dégraderait les performances. Le taux recommandé est $p \in [0.2, 0.4]$. Un taux trop élevé ($p > 0.5$) provoque une perte d'information mémorielle excessive et nuit à l'apprentissage des dépendances longues.

3.8.3. RNN Bidirectionnels

Un RNN bidirectionnel traite la séquence en parallèle dans les deux directions temporelles — de gauche à droite et de droite à gauche — et concatène les états cachés correspondants à chaque position :

$$h_t = [\vec{h}_t ; \overleftarrow{h}_t] \text{ — concaténation avant et arrière, dimension } 2H$$

Cette approche est précieuse pour les tâches d'analyse qui bénéficient du contexte futur : étiquetage de séquences (NER, POS tagging), classification de phrase, encodage pour la traduction. Elle est en revanche inutilisable pour les tâches auto-régressives (génération de texte, décodage en traduction automatique) : à l'inférence, le futur n'est pas encore disponible.

```
rnn = nn.LSTM(input_size=128, hidden_size=256,
             bidirectional=True, batch_first=True)
sortie : (B, T, 2*H) — 2 directions concaténées
```

3.9. Préparation des Données — Tatoeba fra-eng

3.9.1. Description du Dataset

Le corpus Tatoeba fra-eng est un ensemble de paires de phrases bilingues anglais ↔ français. Le corpus complet contient environ 190 000 paires ; nous en utilisons 8 000 après filtrage sur une longueur maximale de 12 tokens par phrase. Les phrases sont courtes et couvrent des sujets quotidiens, avec un vocabulaire relativement limité. Ce dataset constitue un banc d'essai idéal pour expérimenter les architectures Seq2Seq à faible coût computationnel.

3.9.2. Pipeline de Prétraitement

Le pipeline de prétraitement comporte six étapes successives, chacune indispensable à la qualité de l'entraînement :

- Normalisation : conversion unicode → ASCII, mise en minuscules, insertion d'espaces autour de la ponctuation pour garantir une tokenisation cohérente
- Tokenisation : découpage par espace (word-level) — approche simple et suffisante pour ce corpus
- Construction du vocabulaire : dictionnaire bidirectionnel token ↔ index, construit séparément pour le français (source) et l'anglais (cible)
- Tokens spéciaux : quatre tokens réservés avec des rôles précis
- Encodage : conversion de chaque séquence de tokens en liste d'entiers, suivie de l'ajout du token EOS
- Padding et masquage : alignement des séquences d'un même batch sur la longueur maximale, avec création d'un masque booléen

Token	Index	Rôle dans l'architecture
<PAD>	0	Rembourrage — aligne les séquences de longueurs différentes dans le batch
<SOS>	1	Start of Sequence — signal de démarrage pour le décodeur
<EOS>	2	End of Sequence — signal d'arrêt de la génération
<UNK>	3	Unknown — remplace les tokens absents du vocabulaire (OOV)

Tableau 28 : Tokens spéciaux utilisés dans l'architecture du modèle

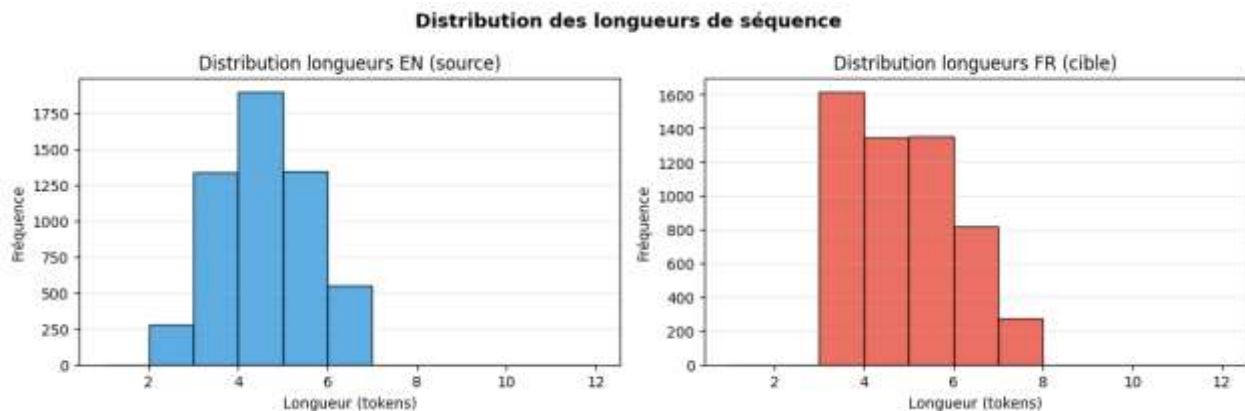


Figure 18 : Distribution des longueurs de séquences (EN/FR)

La figure présente la distribution des longueurs des séquences en **anglais (source)** et en **français (cible)** après la phase de prétraitement et de tokenisation. Les histogrammes montrent que la majorité des phrases contiennent entre **3 et 6 tokens**, avec une concentration autour de **4 à 5 tokens**. Les séquences en français apparaissent légèrement plus longues que celles en anglais, ce qui est cohérent avec le fait qu'une traduction française nécessite souvent davantage de mots pour exprimer une même idée.

Cette distribution indique que le corpus est constitué de **phrases courtes et relativement homogènes**, ce qui facilite l'entraînement des modèles récurrents (RNN, GRU et LSTM). En effet, des séquences de longueur limitée réduisent les coûts de calcul, limitent les problèmes liés aux dépendances très longues et permettent un traitement plus efficace lors de l'apprentissage. Ces observations ont également servi à définir

une **longueur maximale de séquence adaptée**, afin de minimiser le padding tout en conservant la quasi-totalité des données du corpus.

3.9.3. Padding, Masquage et Perte Masquée

Les phrases d'un même mini-lot ont des longueurs différentes. On les aligne sur la longueur maximale du batch en ajoutant des tokens <PAD> à droite. Le masque de padding ($m_t = 1$ si position valide, 0 si PAD) est ensuite utilisé pour ignorer les positions PAD dans le calcul de la perte :

$$L_{\text{masquée}} = \frac{\sum_t m_t \ell_t}{\sum_t m_t}$$

Sans masquage, le modèle gaspillerait de la capacité à prédire des tokens PAD, ce qui dégraderait les performances et biaiserait les métriques. Le masquage garantit que seules les positions porteuses d'information réelle contribuent au gradient.

3.10. Architecture Seq2Seq — Encodeur-Décodeur

3.10.1. Limitation du Modèle de Langage Simple

Un modèle de langage modélise la distribution marginale $P(w_t | w_{<t})$ sur une unique séquence. La traduction automatique est un problème fondamentalement différent : il faut modéliser la distribution conditionnelle $P(y_1, \dots, y_M | x_1, \dots, x_N)$ — une distribution sur la séquence cible (français) conditionnée à la séquence source (anglais). Les deux séquences ont des longueurs différentes ($N \neq M$ en général) et des vocabulaires entièrement distincts. Un LM simple ne peut pas résoudre ce problème.

3.10.2. Architecture Encodeur-Décodeur

Le modèle Seq2Seq (Sutskever, Vinyals & Le, 2014) décompose le problème en deux phases séparées mais interdépendantes :

- Phase 1 — Encodage : le réseau encodeur (un GRU ou LSTM) lit la séquence source $x = (x_1, \dots, x_N)$ token par token et produit un vecteur de contexte $c = h_N^{\text{enc}}$ (l'état caché final, comprimant toute la séquence source en un vecteur de dimension H)
- Phase 2 — Décodage : le réseau décodeur génère la séquence cible $y = (y_1, \dots, y_M)$ token par token, initialisé avec c et conditionné à chaque pas sur sa propre sortie précédente

La décomposition probabiliste sous-jacente est :

$$P(y | x) = \prod_t P(y_t | y_{<t}, x)$$

Les équations du Seq2Seq avec un décodeur GRU sont :

$$c = h_N^{\text{enc}} \text{ (vecteur de contexte — état final de l'encodeur)}$$

$$s_0 = c, \quad s_t = \text{GRU}_{\text{dec}}(y_{t-1}, s_{t-1}), \quad P(y_t | y_{<t}, x) = \text{softmax}(W s_t)$$

3.10.3. Teacher Forcing

Pendant l'entraînement, deux stratégies s'offrent pour choisir l'entrée du décodeur au pas t :

Stratégie	Entrée du décodeur au pas t	Avantage	Inconvénient
Teacher Forcing	Vrai token y^*_{t-1} (gold)	Convergence rapide et stable	Exposure Bias
Auto-régressif	$\hat{y}_{t-1} = \arg \max P(\cdot)$	Proche de l'inférence réelle	Convergence lente, instable

Tableau 29 : Teacher Forcing vs Auto-régressif

L'Exposure Bias est la pathologie centrale du Teacher Forcing : à l'entraînement, le décodeur est toujours alimenté par la vérité (tokens gold), mais à l'inférence il doit utiliser ses propres prédictions — potentiellement erronées. Cette divergence train/inférence peut provoquer une accumulation d'erreurs. La solution recommandée est le curriculum progressif : le ratio de Teacher Forcing commence à 1.0 (100 % gold) et décroît vers 0.2 au cours de l'entraînement.

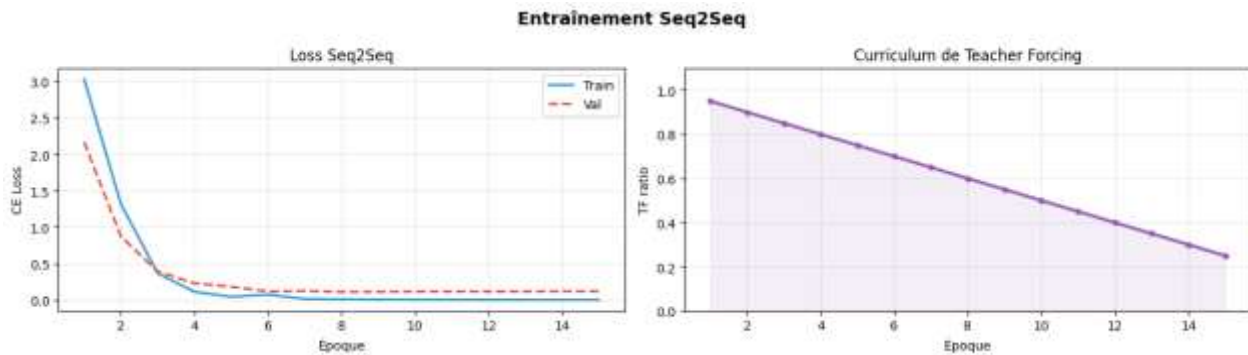


Figure 19 : Entraînement du modèle Seq2Seq et curriculum de Teacher Forcing

La figure présente l'évolution de l'entraînement du modèle **Seq2Seq** à travers deux aspects complémentaires : l'évolution de la fonction de perte (*Cross-Entropy Loss*) et la diminution progressive du **Teacher Forcing Ratio**. Le graphique de gauche montre que les pertes d'entraînement et de validation diminuent rapidement au cours des premières époques, passant d'une valeur supérieure à 3 à une valeur proche de 0,1. Les deux courbes restent très proches tout au long de l'entraînement, ce qui indique une bonne capacité de généralisation et l'absence de surapprentissage significatif. La stabilisation des pertes après quelques époques suggère également une convergence efficace du modèle.

Le graphique de droite illustre la stratégie de **curriculum de Teacher Forcing** adoptée durant l'entraînement. Le **Teacher Forcing Ratio** diminue progressivement de 0,95 à 0,25 au fil des 15 époques. Cette décroissance permet au modèle de s'appuyer fortement sur les séquences cibles réelles au début de l'apprentissage afin de faciliter la convergence, puis d'utiliser progressivement ses propres prédictions pour générer les tokens suivants. Cette approche améliore la robustesse du modèle lors de l'inférence en réduisant l'écart entre les conditions d'entraînement et d'utilisation réelle.

Dans l'ensemble, ces résultats montrent que le modèle Seq2Seq converge rapidement vers une solution stable et que la stratégie de diminution progressive du Teacher Forcing contribue à un apprentissage plus efficace et à une meilleure capacité de généralisation.

3.10.4. Bottleneck du Vecteur de Contexte

La limitation fondamentale du Seq2Seq classique est le bottleneck du vecteur de contexte : toute l'information de la séquence source (potentiellement longue) doit être comprimée en un unique vecteur $c \in \mathbb{R}^H$. Pour les phrases longues, cette compression est destructive — des informations cruciales sont perdues. La solution est le mécanisme d'attention (Bahdanau et al., 2015), qui permet au décodeur d'accéder sélectivement à tous les états cachés de l'encodeur à chaque pas de décodage, plutôt que de dépendre d'un unique résumé fixe.

3.11. Stratégies de Décodage à l'Inférence

3.11.1. Formulation du Problème

À l'inférence, le problème de décodage est la recherche de la séquence cible la plus probable conditionnellement à la source :

$$y^* = \arg \max_y P(y | x) = \arg \max_y \sum_t \log P(y_t | y_{<t}, x)$$

L'espace de recherche est de taille $|V|^M$ — exponentiellement grand — et ne peut pas être parcouru exhaustivement. On utilise des heuristiques de recherche approchée.

3.11.2. Décodage Glouton (Greedy Decoding)

Le décodage glouton sélectionne à chaque pas le token le plus probable selon la distribution de sortie courante :

$$y_t^* = \arg \max_w P(w | y_{<t}^*, x)$$

Sa complexité est $O(T \times |V|)$, ce qui le rend très rapide. Cependant, il est sous-optimal globalement : un excellent premier choix peut conduire à une impasse — une mauvaise séquence partielle dont aucun complément n'est de bonne qualité. Les erreurs sont non récupérables une fois commises.

3.11.3. Beam Search

Le Beam Search généralise le décodage glouton en maintenant les k meilleures hypothèses partielles à chaque étape (k est appelé la largeur du faisceau). Les hypothèses sont scorées par leur log-probabilité cumulée :

$$\text{score}(y_1, \dots, y_t) = \sum_i \log P(y_i | y_{<i}, x)$$

Algorithme du Beam Search :

- Initialiser avec k copies de <SOS>
- À chaque pas, étendre chaque hypothèse avec tous les $|V|$ tokens possibles, calculer les $k \times |V|$ nouveaux scores
- Conserver les k meilleures hypothèses étendues
- Terminer quand toutes les hypothèses ont atteint <EOS> ou la longueur maximale

Pour éviter de favoriser artificiellement les séquences courtes (dont la log-probabilité cumulée est mécaniquement plus élevée), la pénalisation de longueur introduite par Google NMT est recommandée :

$$\text{score normalisé} = (\sum_t \log P(y_t | \dots)) / T^\alpha \text{ avec } \alpha \in [0.6, 0.8]$$

Stratégie	Complexité	Qualité (BLEU)	Vitesse
Glouton (k=1)	$O(T \times V)$	Sous-optimal	Très rapide
Beam Search k=3	$O(T \times 3 \times V)$	Bon compromis	3× plus lent
Beam Search k=5	$O(T \times 5 \times V)$	Meilleure qualité	5× plus lent
Recherche exhaustive	$O(V ^T)$	Optimal	Totalement impraticable

Tableau 30 : Comparaison des stratégies de décodage (Glouton vs Beam Search)

En pratique, $k = 3$ à 5 offre le meilleur compromis qualité/vitesse pour la traduction neuronale. Au-delà de $k = 10$, les gains de qualité deviennent marginaux.

3.12. Évaluation — Perplexité et Score BLEU

3.12.1. Deux Types de Métriques d'Évaluation

L'évaluation des modèles séquentiels s'appuie sur deux classes de métriques complémentaires :

- Métriques intrinsèques (indépendantes de la tâche) : la perplexité mesure la qualité du modèle de langage en tant que tel, sans référence à une tâche applicative. Elle peut être calculée sans annotations humaines supplémentaires.
- Métriques extrinsèques (spécifiques à la tâche) : le score BLEU mesure la qualité de la traduction par comparaison avec des références humaines. Elle requiert des traductions de référence annotées par des experts.

3.12.2. Score BLEU — Bilingual Evaluation Understudy

Le BLEU (Papineni et al., ACL 2002) est la métrique standard de la traduction automatique. Il mesure la précision des n-grammes entre l'hypothèse produite par le modèle et la ou les références humaines, avec une pénalité pour les traductions trop courtes (brevity penalty) :

$$\text{BLEU} = \text{BP} \times \exp(\sum_n w_n \log p_n) \text{ avec } w_n = 1/4 \text{ (BLEU-4)}$$

$$BP = 1 \text{ si } c > r, \exp\left(1 - \frac{r}{c}\right) \text{ si } c \leq r \text{ (pénalité de brèveté)}$$

où p_n est la précision clipée des n -grammes (qui empêche la triche par répétition du même n -gramme), c est la longueur de l'hypothèse et r est la longueur de référence. Le BLEU-4 (n allant de 1 à 4) est la variante la plus utilisée.

3.12.3. Limitations et Métriques Modernes

Le BLEU présente plusieurs limitations importantes qu'il faut connaître : il ne capture pas la sémantique (voiture et automobile sont considérés comme entièrement différents), il ne mesure pas la fluidité grammaticale directement, et il suppose une correspondance lexicale exacte avec la référence. Les métriques modernes comme BERTScore et COMET répondent à ces limitations en utilisant des embeddings contextuels pour comparer l'hypothèse et la référence dans un espace sémantique continu, montrant une bien meilleure corrélation avec les jugements humains.

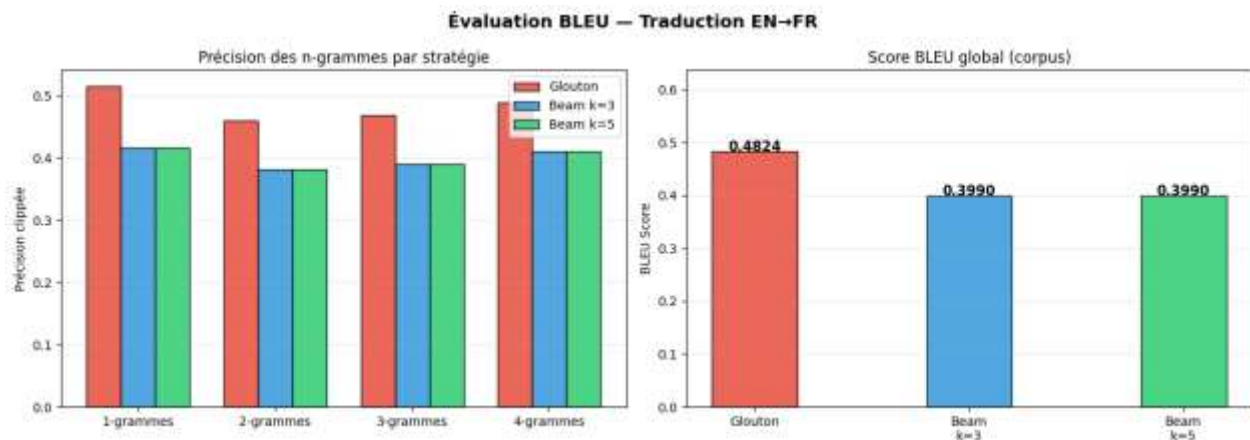


Figure 20 : Précision des n -grammes et score BLEU par stratégie de décodage

La figure présente les performances du modèle de traduction **anglais** → **français** évaluées à l'aide du **score BLEU**, en comparant trois stratégies de décodage : **Glouton (Greedy Search)**, **Beam Search ($k = 3$)** et **Beam Search ($k = 5$)**. Le graphique de gauche montre la précision des **1-grammes** à **4-grammes** pour chaque stratégie. On observe que la stratégie gloutonne obtient systématiquement les meilleures précisions sur l'ensemble des n -grammes, avec des valeurs d'environ **0,51** pour les 1-grammes et **0,48** pour les 4-grammes. Les deux variantes du Beam Search présentent des performances légèrement inférieures mais très proches l'une de l'autre.

Le graphique de droite compare les **scores BLEU globaux** obtenus sur l'ensemble du corpus. Le décodage glouton atteint un score de **0,4824**, tandis que le **Beam Search ($k = 3$)** et le **Beam Search ($k = 5$)** obtiennent chacun un score de **0,3990**. Contrairement aux attentes théoriques, l'augmentation de la largeur du faisceau n'apporte ici aucun gain de performance, les deux configurations produisant exactement le même résultat.

Ces observations indiquent que, pour ce corpus constitué de phrases courtes et relativement simples, le **décodage glouton** est le plus efficace. Il fournit les meilleures traductions tout en étant plus rapide et moins coûteux en calcul que le Beam Search. Les résultats suggèrent également que la complexité supplémentaire introduite par une recherche par faisceau plus large n'apporte pas d'amélioration dans ce contexte expérimental.

3.13. Comparaison Globale des Architectures

Modèle	Atouts principaux	Limitations
RNN Vanilla	Léger, conceptuellement simple, base pédagogique	Gradient évanescent, mauvaise mémoire longue terme
LSTM	Excellent contrôle mémoire, robuste sur longues séquences	4× plus de paramètres, entraînement plus lent
GRU	Bon compromis précision/vitesse, 25% moins de params	Légèrement moins expressif sur très longues dépendances
RNN Profond (L>1)	Représentations hiérarchiques, abstraction croissante	Coût computationnel accru, entraînement plus délicat
RNN Bidirectionnel	Utilise passé ET futur, excellent pour l'encodage	Inadapté au décodage causal et à la génération
Seq2Seq	Modélise proprement la traduction $P(y x)$	Bottleneck du vecteur de contexte fixe, sans attention
Beam Search (k>1)	Meilleur BLEU que le décodage glouton	k× plus lent, sensible au réglage de α et k

Tableau 31 : Comparaison globale des architectures (RNN, LSTM, GRU, Seq2Seq...)

Ce tableau synthétise les compromis fondamentaux entre les différentes architectures. Le choix d'un modèle dépend de la tâche (génération vs encodage), des contraintes computationnelles (paramètres, vitesse d'inférence), et de la longueur typique des séquences à traiter.

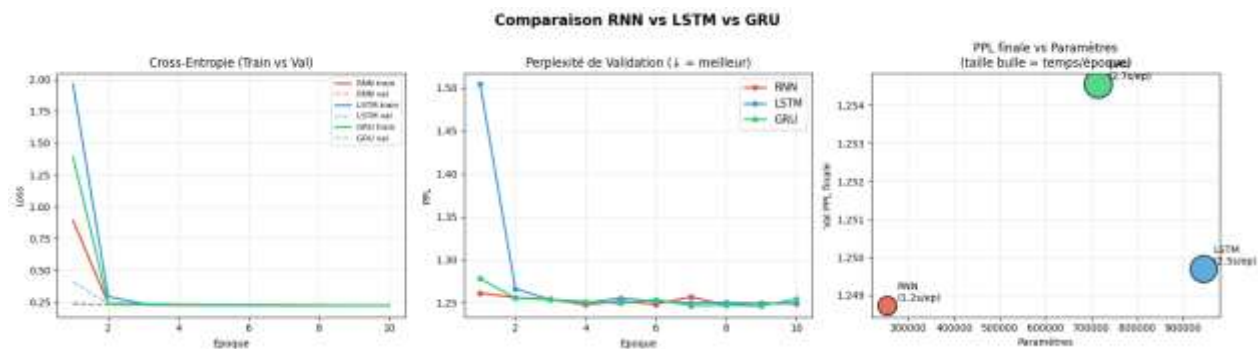


Figure 21 : Comparaison complète RNN vs LSTM vs GRU

La figure compare les performances des trois architectures récurrentes (**RNN Vanilla**, **LSTM** et **GRU**) selon trois critères : l'évolution de la fonction de perte (*Cross-Entropie*), la **perplexité de validation (PPL)** et le compromis entre **complexité paramétrique**, **qualité du modèle** et **temps d'entraînement**. Le premier graphique montre que les trois modèles convergent rapidement au cours des premières époques. Les courbes d'entraînement et de validation restent très proches, indiquant un apprentissage stable et une bonne capacité de généralisation. Après quelques époques, les pertes se stabilisent autour de **0,25**, sans signe de surapprentissage.

Le deuxième graphique présente l'évolution de la **perplexité de validation**, où une valeur plus faible traduit une meilleure qualité de prédiction. Les trois architectures atteignent des performances très similaires, avec une perplexité finale proche de **1,25**. Le LSTM démarre avec une perplexité initiale plus élevée, mais converge rapidement vers le même niveau que les modèles RNN et GRU.

Le troisième graphique met en évidence le compromis entre les performances obtenues et la complexité des modèles. Le **RNN Vanilla** est l'architecture la plus légère (**254 270 paramètres**) et la plus rapide à entraîner ($\approx 1,2$ s/époque), tout en obtenant la meilleure perplexité finale ($\approx 1,249$). Le **GRU** représente un compromis intéressant avec **715 070 paramètres** et un temps d'entraînement intermédiaire ($\approx 1,75$ s/époque), tandis que le **LSTM**, bien qu'étant le modèle le plus complexe (**945 470 paramètres**) et le plus coûteux en temps d'entraînement ($\approx 2,5$ s/époque), n'apporte pas d'amélioration notable de la perplexité.

Ces résultats montrent que, pour ce jeu de données composé de phrases courtes, les architectures plus complexes (**GRU** et **LSTM**) n'offrent pas d'avantage significatif par rapport au **RNN Vanilla**. Ce dernier constitue donc le meilleur compromis entre **qualité de traduction**, **coût computationnel** et **efficacité d'entraînement** dans le cadre de cette étude.

3.14. Synthèse Transversale — Du RNN au Seq2Seq

14.1 Du RNN Vanilla au LSTM/GRU — Résoudre le Problème de Mémoire

La règle de chaîne $P(w) = \prod_t P(w_t | w_{<t})$

définit la tâche de modélisation de séquence : prédire le prochain token conditionnellement à tout le contexte passé. Le RNN implémente cette récurrence via

$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$. En théorie, h_t encode l'intégralité de l'historique $w_1, \dots, w_t$. En pratique, le gradient évanescence l'en empêche : les informations distantes de plus de quelques pas de temps sont effectivement oubliées.

Le LSTM répond à cette limitation via son état de cellule c_t et sa mise à jour additive : $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. Cette opération additive crée une autoroute de gradient : le gradient $\frac{\partial c_t}{\partial c_k}$ n'implique plus un produit de matrices mais un produit de valeurs de porte d'oubli, potentiellement proches de 1. Les expériences empiriques confirment systématiquement la supériorité du LSTM et du GRU sur le RNN vanilla pour les séquences longues (Hochreiter & Schmidhuber, 1997 ; Cho et al., 2014).

3.14.1. Du Modèle de Langage au Seq2Seq — Généraliser à Deux Séquences

La traduction automatique exige $P(y | x)$ — une distribution conditionnelle sur deux espaces différents avec des longueurs asymétriques. L'architecture encodeur-décodeur découple proprement les deux phases : compréhension de la source ($x \rightarrow c$) et génération de la cible ($c, y_{<t} \rightarrow y_t$). Le Teacher Forcing accélère l'entraînement en guidant le décodeur avec les vrais tokens gold, mais crée un écart train/inférence (exposure bias) qui peut dégrader la qualité à l'inférence. Le curriculum progressif (TF : 1.0 $\rightarrow$ 0.2) est la solution empiriquement robuste.

3.14.2. Qualité du Décodage — Beam Search vs Glouton

Le Beam Search améliore systématiquement le score BLEU par rapport au décodage glouton sur des séquences de longueur modérée. Sur un petit corpus comme Tatoeba, le bottleneck principal est la qualité du modèle encodeur-décodeur lui-même, pas la stratégie de décodage. $k = 3$ offre généralement le meilleur compromis qualité/coût computationnel.

3.14.3. Limites et Perspectives — Vers les Transformers

Malgré leurs apports considérables, les architectures récurrentes présentent deux limitations structurelles fondamentales qui ont motivé le développement des Transformers (Vaswani et al., 2017) :

- Bottleneck du vecteur de contexte : compresser une séquence source entière en un unique vecteur fixe $c \in \mathbb{R}^H$ est destructif pour les longues phrases. Le mécanisme d'attention de Bahdanau (2015) résout ce problème en permettant au décodeur d'accéder sélectivement à tous les états cachés de l'encodeur.
- Séquentialité fondamentale : le RNN, le LSTM et le GRU traitent les tokens un par un, de façon strictement séquentielle. Il est impossible de paralléliser sur la dimension temporelle, ce qui limite fortement l'efficacité sur GPU. Le Transformer (Vaswani et al., 2017) supprime complètement la récurrence au profit du mécanisme d'auto-attention (self-attention), permettant un traitement entièrement parallèle.
- Exposure bias : le curriculum learning, le Scheduled Sampling et le Minimum Risk Training sont des solutions partielles, mais aucune n'élimine complètement le problème.

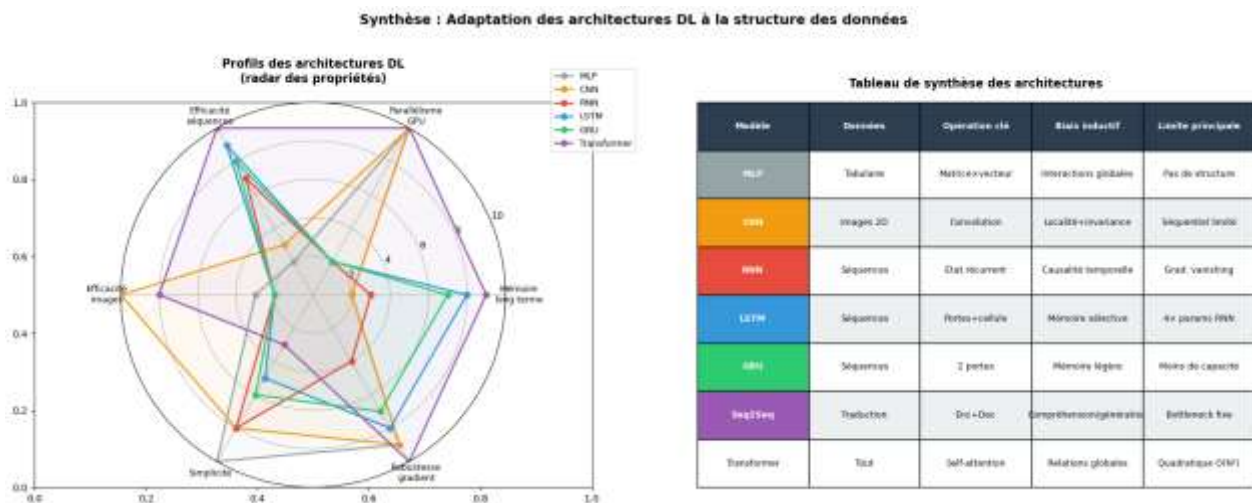


Figure 22 : Synthèse finale : profils radar et tableau comparatif des architectures

Cette figure propose une analyse comparative complète des principales architectures de Deep Learning (DL) en fonction de leurs propriétés et de leur adéquation avec la structure des données. Le visuel se compose de deux parties : un graphique radar évaluant leurs performances selon plusieurs critères, complété par une analyse synthétique de leurs caractéristiques techniques (données, opérations clés, biais inductifs et limites).

1. Analyse des profils de performance (Graphique radar)

Le graphique radar évalue six architectures majeures (MLP, CNN, RNN, LSTM, GRU, Transformer) sur une échelle de 0 à 10 à travers six propriétés fondamentales :

- **Efficacité sur les séquences et mémoire long terme** : Le **Transformer** domine largement sur ces deux aspects, talonné par les architectures à portes comme le **LSTM** et le **GRU** qui gèrent efficacement une mémoire sélective. Les RNN classiques montrent rapidement leurs limites, tandis que les MLP et CNN ne sont pas adaptés.
- **Parallélisme GPU et robustesse du gradient** : Le **Transformer** et le **CNN** maximisent l'utilisation du parallélisme GPU et affichent une excellente stabilité face aux gradients. À l'inverse, la nature séquentielle des modèles récurrents (RNN, LSTM, GRU) freine le parallélisme et les rend plus vulnérables aux problèmes de gradient (notamment le RNN classique).
- **Efficacité sur les images** : Le **CNN** s'impose comme le leader incontesté pour le traitement d'images 2D, le Transformer restant une alternative viable mais plus complexe, tandis que les autres modèles sont inefficaces dans ce domaine.
- **Simplicité** : Le **MLP** (Perceptron Multicouche) se distingue comme l'architecture la plus simple et directe, suivi par le RNN et le CNN. Les modèles plus sophistiqués (LSTM, GRU, Transformer) affichent logiquement un score de simplicité plus faible en raison de leur architecture lourde.

2. Synthèse des caractéristiques techniques et limites

L'analyse de la figure permet de regrouper les spécificités de chaque modèle comme suit :

- **MLP (Perceptron Multicouche)** : Principalement dédié aux données **tabulaires**, il repose sur l'opération **matrice × vecteur**. Son biais inductif est basé sur des **interactions globales**, mais sa limite majeure réside dans son **absence de prise en compte de la structure** (spatiale ou temporelle) des données.
- **CNN (Réseau de neurones convolutif)** : Spécialisé dans les **images 2D**, sa force repose sur la **convolution**, offrant un biais inductif de **localité et d'invariance**. Sa limite principale réside dans un **parallélisme séquentiel limité** sur certains aspects.
- **RNN (Réseau de neurones récurrent)** : Conçu pour les **séquences**, il utilise un **état récurrent** pour modéliser la **causalité temporelle**. Il souffre cependant d'un défaut majeur : la **disparition du gradient** (*vanishing gradient*), ce qui bloque l'apprentissage sur de longues séquences.
- **LSTM** : Amélioration du RNN pour les **séquences**, il introduit des **portes et une cellule** pour une **mémoire sélective**. Sa principale contrainte est son coût de calcul, nécessitant **4 fois plus de paramètres** qu'un RNN classique.
- **GRU** : Alternative au LSTM pour les **séquences**, il simplifie l'architecture à **2 portes** pour une **mémoire plus légère**, mais offre en contrepartie une **capacité de stockage légèrement inférieure** au LSTM.
- **Seq2Seq** : Principalement utilisé en **traduction**, il lie un **Encodeur et un Décodeur** pour la **compréhension et la génération**. Son point faible est son **goulot d'étranglement (bottleneck) fixe**, qui limite le transfert d'informations longues.
- **Transformer** : Modèle universel adapté à **tout type de données**, il repose sur le mécanisme de **self-attention** pour capturer des **relations globales**. Sa limite majeure est sa **complexité quadratique** $\mathcal{O}(N^2)$, qui le rend très gourmand en ressources informatiques à mesure que la taille des données augmente.

3.15. Conclusion

Les RNN, LSTM et GRU ont permis d'améliorer le traitement des données séquentielles, tandis que les modèles Seq2Seq ont ouvert la voie à des applications comme la traduction automatique. Malgré leurs

limites sur les longues séquences, ces architectures restent essentielles pour comprendre les fondements du Deep Learning appliqué au traitement du langage.

Conclusion Générale

Le Deep Learning occupe aujourd'hui une place centrale dans le développement des systèmes d'intelligence artificielle modernes. La diversité des problématiques rencontrées dans des domaines tels que la classification, la vision par ordinateur ou le traitement automatique du langage a conduit à l'émergence d'architectures spécialisées, chacune répondant à des besoins spécifiques. Dans ce contexte, ce projet avait pour objectif d'étudier, d'implémenter et de comparer trois grandes familles de réseaux de neurones : les perceptrons multicouches (MLP), les réseaux de neurones convolutionnels (CNN) et les réseaux de neurones récurrents (RNN).

La première partie a permis d'aborder les fondements du Deep Learning à travers les MLP appliqués à des données tabulaires. Cette étude a mis en évidence les mécanismes essentiels de l'apprentissage supervisé, notamment la propagation avant, la rétropropagation du gradient et l'optimisation des paramètres du modèle. Bien qu'efficaces sur des données vectorisées, les MLP présentent certaines limites lorsqu'ils sont confrontés à des données possédant une structure spatiale ou temporelle complexe.

La deuxième partie, consacrée aux CNN, a démontré l'intérêt des opérations de convolution et de pooling dans le traitement des images. Les expérimentations réalisées sur le jeu de données Fashion-MNIST ont montré que ces architectures sont capables d'extraire automatiquement des caractéristiques pertinentes, conduisant à des performances supérieures à celles des approches entièrement connectées dans les tâches de classification visuelle. L'analyse des cartes de caractéristiques et l'étude des hyperparamètres ont également permis de mieux comprendre le fonctionnement interne des modèles convolutionnels.

Enfin, la troisième partie a exploré les architectures dédiées aux données séquentielles. L'étude des RNN, des LSTM et des GRU a mis en évidence les défis liés à la modélisation des dépendances temporelles ainsi que les solutions proposées pour atténuer les problèmes de disparition ou d'explosion du gradient. La mise en œuvre d'un modèle Seq2Seq appliqué à une tâche de traduction automatique a illustré la capacité des réseaux récurrents à traiter des séquences de longueur variable et à générer des sorties cohérentes. Les métriques d'évaluation, telles que la perplexité et le score BLEU, ont permis d'apprécier objectivement la qualité des modèles développés.

Au-delà des résultats obtenus, ce projet a permis de renforcer des compétences essentielles en Deep Learning, notamment en conception de modèles, en utilisation de PyTorch et en analyse des performances. Les expérimentations réalisées ont montré que le choix d'une architecture dépend fortement de la nature des données et des objectifs visés. En perspective, ce travail pourrait être enrichi par l'étude d'architectures plus avancées, telles que les ResNet ou les Transformers, afin d'approfondir la compréhension des techniques modernes d'intelligence artificielle.

Références

- **Cybenko (1989)** : démonstration du théorème d'approximation universelle pour les réseaux à une couche cachée.
- **Hornik (1991)** : extension du théorème d'approximation universelle à des classes plus générales de réseaux de neurones.
- **LeCun et al. (1989)** : première démonstration empirique de l'efficacité des réseaux de neurones convolutionnels.
- **LeCun et al. (1998)** : proposition de l'architecture **LeNet-5** pour la reconnaissance automatique des chiffres manuscrits.
- **Srivastava et al. (2014)** : introduction de la technique de régularisation **Dropout**.
- **Glorot & Bengio (2010)** : développement de la méthode d'initialisation des poids **Xavier**.
- **Ioffe & Szegedy (2015)** : introduction de la **Batch Normalization** pour stabiliser et accélérer l'entraînement.
- **Kingma & Ba (2014/2015)** : proposition de l'optimiseur **Adam**, largement utilisé en Deep Learning.
- **Xiao et al. (2017)** : création du jeu de données **Fashion-MNIST** comme alternative à MNIST.
- **Hubel & Wiesel (1981)** : travaux sur les cellules simples du cortex visuel V1, ayant inspiré les CNN (Prix Nobel de physiologie ou médecine).
- **Hochreiter & Schmidhuber (1997)** : introduction des réseaux **LSTM (Long Short-Term Memory)**.
- **Cho et al. (2014)** : proposition des unités récurrentes **GRU (Gated Recurrent Unit)**.
- **Sutskever, Vinyals & Le (2014)** : développement de l'architecture **Seq2Seq (Sequence-to-Sequence)**.
- **Bahdanau et al. (2015)** : introduction du **mécanisme d'attention** dans les modèles Seq2Seq.
- **Vaswani et al. (2017)** : proposition des **Transformers**, fondés sur le mécanisme de self-attention.
- **Papineni et al. (2002)** : développement du score **BLEU** pour l'évaluation automatique des traductions.
- **Google Neural Machine Translation (GNMT)** : introduction de la **pénalisation de longueur** pour améliorer le Beam Search en traduction automatique.